

Poole Defect Calculus

Hysteretic Surface Selection, Planar Defects, Pore/Crack/Hole Laws, Channel Spectra, Prime Corrections, Work Laws, Q/P Boundaries, and Empirical Atlases

Rooke Alan Poole

June 2026

Abstract

This paper presents the Poole Defect Calculus as a discrete mathematical theory of local support, surface selection, and defect response in the three-dimensional binary cellular automaton $B5-7/S5-9$ on a periodic cubic lattice with the 26-neighbor Moore neighborhood. The update rule can be written as the compact hysteretic support law

$$x_{t+1}(i) = \mathbf{1}\{5 \leq N_i(x_t) \leq 7 + 2x_t(i)\}.$$

The term $2x_t(i)$ is not an added field; it is a state-dependent widening of the upper support window for active cells. From this single local rule one obtains exact results for stable one-cell sheets, surface selection, finite propagation, non-monotonicity, non-reversibility, planar defect response, pore stability, crack and line-hole behavior, square and rectangle puncture laws, channel spectra, prime-corrected response, first-step irreversible work, and a limited Q/P-style rectangle reconstruction boundary. The theorem-level center of the paper is the one-step planar defect selector: for a defective stable sheet, first response is determined by the open-defect and closed-defect counts q_D and r_D . Rectangular defects form a closed zero-death subfamily with

$$B_0(a, b) = 4a + 4b + 12 = 2P + 12,$$

channel spectrum $(N_5, N_6, N_7) = (12, 4a + 4b - 16, 16)$, prime-corrected response $B_\phi(a, b) = 4a + 4b - 4$, and exact first-step work laws. Empirical atlas material is included only as law-discovery evidence for multi-step relaxation and non-planar void response; it is not treated as a closed theorem.

License: Apache License 2.0 / SPDX-License-Identifier: Apache-2.0

Contents

1 Purpose and result status	5
2 The finite Poole lattice	5
3 The compact hysteretic support law	5
4 Equivalent normal forms of the update rule	6
4.1 Birth-survival form	6
4.2 Capacity form	6
4.3 Deficit-excess form	6
4.4 Channel form	7
5 Support strain and local event channels	7
6 Expanded diagnostic vocabulary	8
7 Basic structural theorems	8
8 Counterexample library for structural safety	9
8.1 Monotonicity failure	9
8.2 Reversibility failure	9
8.3 Mean-field caution	9
9 Fixed points and active graph constraints	10

10 Surface selection	10
11 Dimensional selector derivation	11
12 Planar defect calculus	11
13 Planar operator table and worked examples	13
13.1 Single pore	13
13.2 Separated pore gas	13
13.3 Cracks and line sources	13
14 Square, rectangle, and line-hole laws	13
14.1 Square and rectangle punctures	13
15 Expanded rectangle morphology	14
15.1 Plane contribution	14
15.2 Normal halo contribution	14
15.3 Channel morphology	15
15.4 Overcompensation	15
15.5 Line-hole edge case	15
16 Model tags: raw PM_0 and prime-sharpened PM_ϕ	15
17 Rectangle channel spectrum and prime correction	16
18 Irreversible work laws for rectangle defects	16
19 Model-tagged arbitrary planar selector	17
20 Q/P-style rectangle reconstruction boundary	18
21 Reconstruction algorithm under a rectangle prior	18
22 Verification and empirical ledgers	19
22.1 v7 perimeter audit	19
22.2 v9 theorem-level closure	19
22.3 v10 empirical atlas	20
23 Mean-field failure and correlation dependence	20
24 Continuum expansion of the 26-neighbor sum	20
25 Rule-space interpretation	21
26 Limitations and open problems	21
27 Conclusion	22
Appendices	23

A.1	Minimal implementation requirements	23
A.2	Pseudocode	23
A.3	Theorem ledger	23
A.4	Complete theorem and claim-status ledger	25
A.5	Model dictionary	26
A.6	Defect operator cookbook	26
A.6.1	General nearby-rule selector	27
A.7	Detailed rectangle morphology and channel count	27
A.7.1	Work values	28
A.8	Rectangle reconstruction boundary expanded	28
A.9	Verification ledger	28
A.10	Expanded v10 empirical atlas	29
A.10.1	v10 audit configuration	29
A.10.2	Planar multi-step relaxation summary	29
A.10.3	Mean 3D void response table	30
A.11	Reference implementation appendix	30
A.12	Two-dimensional defect selector reference implementation	32
A.13	Mean-field closure and why it fails structurally	33
A.14	Continuum expansion caveat	33
A.15	Glossary	34
A.16	Open theorem roadmap	34
A.17	Final boundary statement	35
A.18	Publication-form proof capsules	36
A.18.D01:	stable active plane	36
A.18.D02:	surface selector	36
A.18.D03–D05:	square and rectangle response	36
A.18.D06:	line-hole edge law	36
A.18.D07–D08:	channel spectrum and prime correction	36
A.18.D09–D10:	irreversible work	37
A.18.D11:	arbitrary planar selector	37
A.18.D12–D13:	rectangle reconstruction boundary	37
A.19	Full reference verifier script	37

A.20	Empirical atlas reference harness	42
A.21	CSV schema appendix	46
A.22	Claim boundaries and overclaim controls	46
	Evidence and Verification Appendices	47
A.23	Evidence role	47
A.24	Pass/fail summary	47
A.25	Rectangle, square, channel, prime, and overcompensation audit	47
A.26	Pin-hole edge audit	53
A.27	Arbitrary planar selector audit	55
A.28	/P-style rectangle reconstruction audit	57
A.29	Empirical multi-step trace table	58
A.30	Evidence file schema	60
A.31	Evidence checksum manifest	61
A.32	Evidence reference verifier script	62
	Claim-Boundary and Language Appendices	74
A.33	Role of claim-boundary controls	74
A.34	Hardened claim-map table	74
A.35	Hardening by claim-status lane	75
A.36	Language audit and overclaim controls	76
A.37	Theorem-level statements that are safe to foreground	77
A.38	Statements that must stay bounded	77

1 Purpose and result status

The Poole Defect Calculus is best read as a discrete mathematical theory first. It studies how local support thresholds respond to missing cells, punctures, pores, cracks, rectangular holes, and three-dimensional voids inside a deterministic cellular automaton. It does not by itself prove a physical theory of spacetime, gravity, quantum capture, or cosmology. Those possible interpretations belong to later bridge documents.

The claim hierarchy used throughout this paper is:

- (i) **Theorem-level claims:** consequences of the local update rule or finite combinatorics.
- (ii) **Audit-level claims:** results from explicit finite simulations or verification scripts.
- (iii) **Empirical-atlas claims:** reproducible numerical observations that suggest candidate laws but are not proofs.
- (iv) **Bridge claims:** possible interpretations in OTG, Q/P capture, irreversibility, or continuum geometry. These are noted only as boundaries, not as established results.

The central mathematical message is narrow and strong: the Poole rule is a hysteretic surface dynamics whose first-step response to planar defects is an exact local selector.

2 The finite Poole lattice

Definition 2.1 (Periodic cubic lattice). *Fix $L \in \mathbb{N}$. The finite Poole lattice is*

$$\Lambda_L = (\mathbb{Z}/L\mathbb{Z})^3.$$

A configuration is a map $x : \Lambda_L \rightarrow \{0, 1\}$. The value $x(i) = 1$ means that the voxel i is active; $x(i) = 0$ means inactive.

Definition 2.2 (Moore neighborhood). *The radius-one 26-neighbor Moore neighborhood is*

$$\mathcal{N}_{26} = \{-1, 0, 1\}^3 \setminus \{(0, 0, 0)\}.$$

The active neighbor count at site i is

$$N_i(x) = \sum_{\delta \in \mathcal{N}_{26}} x(i + \delta),$$

where addition is modulo L in each coordinate.

The periodic boundary condition eliminates boundary artifacts. It also makes the update rule equivariant under translations, coordinate permutations, and coordinate sign flips.

3 The compact hysteretic support law

The raw Poole rule $B5-7/S5-9$ is:

- an inactive cell is born if it has 5, 6, or 7 active neighbors;
- an active cell survives if it has 5, 6, 7, 8, or 9 active neighbors.

Definition 3.1 (Base Poole map). *The base Poole map $\mathcal{P} : \{0, 1\}^{\Lambda_L} \rightarrow \{0, 1\}^{\Lambda_L}$ is defined coordinatewise by*

$$\mathcal{P}(x)_i = \mathbf{1}\{5 \leq N_i(x) \leq 7 + 2x(i)\}. \quad (1)$$

The evolution is $x_{t+1} = \mathcal{P}(x_t)$.

Theorem 3.1 (Compact-rule equivalence). *The compact law (1) is exactly equivalent to B5-7/S5-9.*

Proof. If $x(i) = 0$, then $7 + 2x(i) = 7$, so the condition is $5 \leq N_i(x) \leq 7$, which is the birth rule. If $x(i) = 1$, then $7 + 2x(i) = 9$, so the condition is $5 \leq N_i(x) \leq 9$, which is the survival rule. $\square$

Definition 3.2 (State-dependent support capacity). *The upper support capacity of site i in configuration x is*

$$C_i(x) = 7 + 2x(i).$$

Inactive sites have capacity 7; active sites have capacity 9.

Remark 3.1 (Local hysteresis). The automaton is Markovian in the full configuration, but locally hysteretic in the decision at a site. The current state of the site changes the upper threshold used to decide the next state. Active structure receives a two-neighbor persistence allowance.

4 Equivalent normal forms of the update rule

The compact law is the most concise form of the rule, but the release should preserve several equivalent views because each one is useful for a different proof.

4.1 Birth-survival form

The original local rule is

$$x_{t+1}(i) = (1 - x_t(i))\mathbf{1}\{N_i \in \{5, 6, 7\}\} + x_t(i)\mathbf{1}\{N_i \in \{5, 6, 7, 8, 9\}\}.$$

This is the most readable implementation form and should be used in the reference verifier.

4.2 Capacity form

Let

$$C_i(x_t) = 7 + 2x_t(i).$$

Then

$$x_{t+1}(i) = \mathbf{1}\{5 \leq N_i(x_t) \leq C_i(x_t)\}.$$

This form exposes hysteresis: the current active state widens the upper threshold by two. Inactive sites are allowed to enter only through the narrow band 5, 6, 7; active sites persist through 8, 9 as well.

4.3 Deficit-excess form

Define local deficit and excess by

$$D_i = (5 - N_i)_+, \quad E_i = (N_i - C_i)_+.$$

Then

$$x_{t+1}(i) = 1 \iff D_i = 0 \text{ and } E_i = 0.$$

This form is useful for support-strain diagnostics and for empirical atlas measurements.

4.4 Channel form

The accepted birth channels are B_5, B_6, B_7 . The accepted survival channels are S_5, S_6, S_7, S_8, S_9 . For rectangle defects the raw first-step births split into a fixed channel spectrum:

$$(N_5, N_6, N_7) = (12, 4a + 4b - 16, 16).$$

The default prime-corrected model removes the raw N_7 birth group while leaving the rectangle N_5 and N_6 groups, so it gives $B_\phi(a, b) = 4a + 4b - 4$.

5 Support strain and local event channels

Definition 5.1 (Support windows). *For state $s \in \{0, 1\}$, define*

$$W_s = [5, 7 + 2s] \cap \mathbb{Z}.$$

Thus $W_0 = \{5, 6, 7\}$ and $W_1 = \{5, 6, 7, 8, 9\}$.

Definition 5.2 (Margin, deficit, excess, and signed strain). *Define*

$$\begin{aligned} m_i(x) &= \min\{N_i(x) - 5, C_i(x) - N_i(x)\}, \\ D_i(x) &= (5 - N_i(x))_+, \\ E_i(x) &= (N_i(x) - C_i(x))_+, \\ \psi_i(x) &= E_i(x) - D_i(x). \end{aligned}$$

Here $(a)_+ = \max\{a, 0\}$. Negative ψ_i means starvation or under-support; positive ψ_i means satiation or overcrowding; zero ψ_i means the local support window is satisfied.

Proposition 5.1 (Update as zero-strain selection). *For the base Poole rule,*

$$\mathcal{P}(x)_i = 1 \iff \psi_i(x) = 0 \iff m_i(x) \geq 0.$$

Proof. The condition $\psi_i(x) = 0$ is equivalent to $D_i(x) = 0$ and $E_i(x) = 0$, hence to $5 \leq N_i(x) \leq C_i(x)$. Substituting $C_i(x) = 7 + 2x(i)$ gives (1). $\square$

Definition 5.3 (Frontier-normalized support energy). *Let*

$$\mathcal{F}(x) = \{i \in \Lambda_L : x(i) = 1 \text{ or } N_i(x) > 0\}.$$

Define

$$\mathcal{E}(x) = \begin{cases} \frac{1}{|\mathcal{F}(x)|} \sum_{i \in \mathcal{F}(x)} \psi_i(x)^2, & |\mathcal{F}(x)| > 0, \\ 0, & |\mathcal{F}(x)| = 0. \end{cases}$$

This energy is diagnostic, not a proven Lyapunov function. It is useful because it separates under-supported, valid, and over-supported sites while keeping the computation local.

Definition 5.4 (Birth and death channels). *A birth event at site i has channel B_S if $x(i) = 0$, $\mathcal{P}(x)_i = 1$, and $N_i(x) = S$. A death event has channel D_S if $x(i) = 1$, $\mathcal{P}(x)_i = 0$, and $N_i(x) = S$.*

The channel B_7 is especially important in the audited regimes. It is the upper edge of the birth window, and in the rectangle laws it is exactly the channel removed by the default prime-sharpened model PM_ϕ . In this paper it is therefore called a *plasticity gate*: it allows shallow over-edge geometry to become active, but it also injects noisy growth in some regimes.

6 Expanded diagnostic vocabulary

The theorem layer can be stated using only neighbor counts, but the release should preserve the diagnostic language because it explains how the empirical tools measure response.

Definition 6.1 (Event count and churn). *For one update $x \mapsto y$, define births, deaths, and total events by*

$$B(x) = |\{i : x(i) = 0, y(i) = 1\}|, \quad D(x) = |\{i : x(i) = 1, y(i) = 0\}|, \\ E(x) = B(x) + D(x).$$

A normalized event density is $E(x)/L^3$, or, in sheet-restricted experiments, $E(x)/L^2$ when the natural object is a one-cell sheet.

Definition 6.2 (Channel-resolved event vector). *The birth-channel vector is*

$$\mathbf{B}(x) = (B_5(x), B_6(x), B_7(x)),$$

where $B_k(x)$ counts inactive sites born with raw neighbor count k . The survival-memory vector is

$$\mathbf{S}_+(x) = (S_8(x), S_9(x)),$$

where $S_k(x)$ counts active sites surviving with raw neighbor count k . The pair $(B_7, S_8 + S_9)$ is useful because B_7 is the high birth edge and S_8, S_9 are the two channels added by hysteretic memory.

Definition 6.3 (First-step work functional). *Let $I_k \geq 0$ be a chosen cost assigned to a birth at raw channel k . The first-step birth work is*

$$W_B(x) = \sum_{k \in \{5,6,7\}} I_k B_k(x).$$

When $I_k = \log 2 + \log \binom{26}{k}$, the weight records the binary event bit plus the local channel multiplicity. Other nonnegative channel weights can be used, but the chosen weights must be stated.

Remark 6.1 (Diagnostic status). Churn, work, strain, and channel flux are measurement tools. The rectangle work laws are exact once the channel spectrum and weights are fixed, but the weights themselves are a chosen bookkeeping model unless separately justified.

7 Basic structural theorems

Proposition 7.1 (Cubic and translation equivariance). *The map $\mathcal{P}$ commutes with translations of Λ_L , coordinate permutations, and coordinate sign flips.*

Proof. The neighborhood $\mathcal{N}_{26}$ is invariant under those operations as a set. Such operations only permute terms in the neighbor sum $N_i(x)$, leaving the update condition unchanged. $\square$

Proposition 7.2 (Finite propagation speed). *If two initial configurations agree on the radius- t Moore ball around i , then their states at i agree after t updates. Influence propagates at speed at most one lattice unit per generation in Chebyshev distance.*

Proof. One update depends only on the radius-one Moore ball. Iterating this dependency gives a radius- t dependency after t updates. $\square$

Proposition 7.3 (Eventual periodicity). *For fixed L , every trajectory under $\mathcal{P}$ is eventually periodic. The preperiod plus period is at most 2^{L^3} .*

Proof. There are 2^{L^3} configurations. A deterministic map on a finite set must eventually revisit a previous state; after a repeat, the future is periodic. $\square$

Proposition 7.4 (Non-monotonicity). *The Poole map is not monotone with respect to coordinatewise order.*

Proof. Let $x \leq y$ and choose a site i with $x(i) = y(i) = 0$. Give x exactly five active neighbors of i . Let y be obtained by adding three additional active neighbors around i , so $N_i(y) = 8$. Then $\mathcal{P}(x)_i = 1$ but $\mathcal{P}(y)_i = 0$, because eight neighbors over-support an inactive site. Hence monotonicity fails. $\square$

Proposition 7.5 (Non-reversibility). *For $L \geq 3$, $\mathcal{P}$ is not injective and therefore is not reversible.*

Proof. The all-zero configuration maps to itself. The all-one configuration has $N_i = 26$ at every site, so every active site is over-supported and dies. Thus the all-one configuration also maps to the all-zero configuration. $\square$

8 Counterexample library for structural safety

A paper should include small counterexamples because they prevent readers from importing false assumptions from monotone cellular automata.

8.1 Monotonicity failure

Choose a target inactive site i . Let configuration x contain exactly five active Moore neighbors of i , and let y contain those five neighbors plus three more, with all other cells equal. Then $x \leq y$, but

$$\mathcal{P}(x)_i = 1, \quad \mathcal{P}(y)_i = 0,$$

with $N_i(x) = 5$ and $N_i(y) = 8$. More active support can destroy birth eligibility because inactive sites have a narrow upper threshold of 7.

8.2 Reversibility failure

The all-zero configuration maps to itself. On a periodic lattice with $L \geq 3$, the all-one configuration has raw count 26 at every site, so every active site is over-supported and dies. Thus two distinct configurations map to the all-zero configuration. The rule is not reversible.

8.3 Mean-field caution

A binomial mean-field closure treats every local neighborhood as if it were independently sampled. That closure is structurally mismatched to sheet-selected dynamics: the stable sheet is not random occupancy, but a correlated two-dimensional object embedded in three dimensions. Mean-field calculations remain useful as baselines precisely because the actual stable structures violate independence.

9 Fixed points and active graph constraints

Definition 9.1 (Active adjacency graph). *For a configuration x , let $G_A(x)$ be the graph whose vertices are active sites and whose edges connect active sites at Moore-neighbor distance.*

Proposition 9.1 (Fixed-point constraints). *A configuration x is a fixed point iff for every site i :*

$$x(i) = 1 \Rightarrow 5 \leq N_i(x) \leq 9,$$

and

$$x(i) = 0 \Rightarrow N_i(x) \notin \{5, 6, 7\}.$$

Proof. A fixed active site must survive, and a fixed inactive site must avoid birth. These are exactly the stated conditions. $\square$

Proposition 9.2 (Active-degree bounds in fixed points). *If x is a nonzero fixed point and $A = \{i : x(i) = 1\}$, then every active vertex has active Moore degree between 5 and 9. Consequently*

$$5 \leq \frac{2|E(G_A)|}{|A|} \leq 9.$$

Proof. For an active site, $N_i(x)$ is its degree in $G_A(x)$. The fixed-point survival condition gives $5 \leq N_i(x) \leq 9$ for every active site. Summing degrees gives the result. $\square$

This immediately rules out persistent isolated points and thin isolated filaments as fixed active components.

10 Surface selection

Definition 10.1 (Coordinate-flat local pattern). *A configuration is locally a coordinate r -flat through a site i if, inside the $3 \times 3 \times 3$ Moore block centered at i , the active sites are exactly the sites in an r -dimensional coordinate subspace through i . The cases $r = 0, 1, 2, 3$ represent a point, line, sheet, and filled block.*

Lemma 10.1 (Flat local degree count). *If an active site is locally a coordinate r -flat, then*

$$N_i = 3^r - 1.$$

Thus the flat local degrees are 0, 2, 8, 26 for $r = 0, 1, 2, 3$.

Proof. The r -flat contributes 3^r sites in the centered Moore block, one of which is the center site itself. The remaining $3^r - 1$ are active neighbors. $\square$

Theorem 10.1 (Local surface selector). *Among coordinate-flat local bulk types, the Poole survival window $5 \leq N_i \leq 9$ preserves exactly the locally two-dimensional sheet type.*

Proof. The four local degrees are 0, 2, 8, 26. Only 8 lies in $[5, 9]$. Hence points and lines are under-supported, sheets survive, and filled bulk is over-supported. $\square$

Theorem 10.2 (Stable active plane). *Assume $L \geq 3$. Fix a coordinate direction and a level c . Let*

$$H_c = \{(a, b, c) : a, b \in \mathbb{Z}/L\mathbb{Z}\} \subset \Lambda_L,$$

with configuration $x = \mathbf{1}_{H_c}$. Then $\mathcal{P}(x) = x$.

Proof. An active site in the sheet has the other eight sites in its in-plane 3×3 block as active neighbors, so $N = 8$ and it survives. A site in either adjacent layer sees a full 3×3 footprint, so $N = 9$; inactive cells are born only for $5 \leq N \leq 7$, so no adjacent birth occurs. Sites farther away see $N = 0$. Therefore the sheet is unchanged. $\square$

Corollary 10.1 (The +2 term is sheet memory). *Under the memory-reduced ablation $B5-7/S5-7$, a full one-cell active plane dies in one step, because its active sites have $N = 8$, outside the survival window 5–7.*

Proposition 10.1 (Filled slabs are rejected). *Let S be a union of at least two consecutive full coordinate sheets. Then every active site has at least 17 active neighbors, so no active site survives. No inactive site has a birth count in $\{5, 6, 7\}$. Thus $\mathcal{P}(\mathbf{1}_S) = 0$.*

Proof. An active site in a slab of thickness at least two sees eight active neighbors in its own sheet and at least nine in an adjacent sheet, for at least $17 > 9$. An inactive site outside the slab sees either no active sheet, one full active sheet, or two full active sheets, giving counts 0, 9, 18, none of which is a birth count. $\square$

11 Dimensional selector derivation

The surface selector theorem can be understood by looking at ideal coordinate flats. In a complete coordinate r -flat embedded in three dimensions, a site sees

$$3^r - 1$$

active neighbors inside the flat. Thus a point-like zero-flat gives 0, a line gives 2, a sheet gives 8, and full bulk gives 26. The survival window $5 \leq N \leq 9$ accepts the sheet and rejects the point, the line, and the bulk.

Local object	Dimension r	Ideal count $3^r - 1$	Survival status
point/dust	0	0	rejected by starvation
line	1	2	rejected by starvation
one-cell sheet	2	8	accepted
filled local bulk	3	26	rejected by over-support

This is the mathematical source of the term “surface selector.” The rule is not merely preserving a particular plane by accident; its threshold window accepts the count signature of a two-dimensional sheet while rejecting lower-dimensional and full-dimensional ideal flats.

Remark 11.1 (Why slabs fail). A filled slab of thickness at least two is not a one-cell surface. Interior or near-interior slab sites see too many active neighbors and violate the upper survival threshold. This is why the calculus should be built around one-cell sheets and their defects, not thick filled plates.

12 Planar defect calculus

The exact defect calculus begins by removing a mask from an otherwise stable sheet.

Definition 12.1 (Defective sheet and defect mask). *Identify the coordinate sheet with the two-dimensional torus*

$$T_L = (\mathbb{Z}/L\mathbb{Z})^2.$$

Let $D \subset T_L$ be a set of missing sheet sites, called the defect mask. The corresponding three-dimensional configuration is

$$x_D(u, c) = \mathbf{1}\{u \notin D\}, \quad x_D(u, z) = 0 \quad \text{for } z \neq c.$$

Definition 12.2 (Open and closed defect counts). *For $u \in T_L$, define*

$$q_D(u) = \sum_{\eta \in \{-1, 0, 1\}^2 \setminus \{0\}} \mathbf{1}\{u + \eta \in D\},$$

$$r_D(u) = \sum_{\eta \in \{-1, 0, 1\}^2} \mathbf{1}\{u + \eta \in D\}.$$

Here q_D counts missing cells in the open eight-neighborhood in the sheet, while r_D counts missing cells in the closed 3×3 footprint.

Theorem 12.1 (One-step planar defect calculus). *For the base Poole rule, the first update of x_D is determined exactly as follows:*

- (a) *an active sheet cell $u \notin D$ survives exactly when $q_D(u) \leq 3$, and dies exactly when $q_D(u) \geq 4$;*
- (b) *an inactive in-plane defect cell $u \in D$ is born in the original sheet plane exactly when $1 \leq q_D(u) \leq 3$;*
- (c) *an inactive cell one normal layer above or below the sheet, at $(u, c + 1)$ or $(u, c - 1)$, is born exactly when $2 \leq r_D(u) \leq 4$;*
- (d) *all layers with $|z - c| \geq 2$ remain inactive after the first update.*

Proof. All active sites initially lie in the sheet. A site in the sheet plane sees only the eight planar neighbors. If $u \notin D$, then

$$N(u, c) = 8 - q_D(u).$$

Survival requires $5 \leq 8 - q_D(u) \leq 9$, equivalent to $q_D(u) \leq 3$. If $u \in D$, the same count applies but the site is inactive, so birth requires $5 \leq 8 - q_D(u) \leq 7$, equivalent to $1 \leq q_D(u) \leq 3$. A site one layer above or below the sheet sees a 3×3 sheet footprint, so

$$N(u, c \pm 1) = 9 - r_D(u).$$

Birth requires $5 \leq 9 - r_D(u) \leq 7$, equivalent to $2 \leq r_D(u) \leq 4$. Sites two or more layers away see no active neighbors on the first update. $\square$

This theorem is the core of the Poole Defect Calculus. It converts a three-dimensional update near a sheet defect into two two-dimensional convolutions on D .

Corollary 12.1 (Stable pinhole gas). *If every closed 3×3 footprint contains at most one defect site, then x_D is a fixed point. In particular, a single one-cell pore in an otherwise complete sheet is exactly stable.*

Proof. The condition gives $r_D(u) \leq 1$ everywhere, so no normal birth can satisfy $2 \leq r_D \leq 4$. Each defect site has $q_D = 0$, so no in-plane birth occurs. Every active sheet site has $q_D \leq 1 \leq 3$, so all active sheet cells survive. $\square$

Remark 12.1 (Defect taxonomy). The exact local selectors give a clean taxonomy. Isolated pores freeze. Slits and cracks act as line sources. Two-dimensional punctures emit symmetric normal halos. General planar holes can cause both births and deaths; rectangles are a special zero-death subfamily.

13 Planar operator table and worked examples

The exact planar defect calculus is the core of the release. Let $D \subset T_L$ be the missing-cell mask in a stable coordinate sheet H_c . For $u \in T_L$, define the open and closed footprints

$$Q(u) = \{u + \delta : \delta \in \{-1, 0, 1\}^2 \setminus \{0\}\}, \quad R(u) = \{u + \delta : \delta \in \{-1, 0, 1\}^2\}.$$

Then

$$q_D(u) = |D \cap Q(u)|, \quad r_D(u) = |D \cap R(u)|.$$

The first response is completely determined by these two numbers.

Cell class	Raw count	Rule test	Result
active sheet site $u \notin D$	$8 - q_D(u)$	survival $5 \leq N \leq 9$	survives iff $q_D(u) \leq 3$
missing in-plane defect $u \in D$	$8 - q_D(u)$	birth $5 \leq N \leq 7$	born iff $1 \leq q_D(u) \leq 3$
normal-layer site $(u, c \pm 1)$	$9 - r_D(u)$	birth $5 \leq N \leq 7$	born iff $2 \leq r_D(u) \leq 4$
site two or more layers away	0	birth impossible	inactive

13.1 Single pore

For a single missing cell, every closed footprint contains at most one defect. Therefore every active sheet site has $q_D \leq 1$ and survives. The missing pore has $q_D = 0$, so it does not rebirth because inactive birth requires $N \leq 7$, while the pore sees 8 active planar neighbors. Normal-layer sites have $r_D \leq 1$, below the accepted range $2 \leq r_D \leq 4$. A single pore is exactly stable.

13.2 Separated pore gas

If every closed 3×3 footprint contains at most one defect, the same argument applies at every pore. The defective sheet is fixed. This is a useful theorem because it shows that defects do not automatically heal; sufficiently isolated pores are locked by over-supported in-plane birth and under-supported normal-layer birth.

13.3 Cracks and line sources

A one-cell-wide slit has contiguous missing cells. Its endpoints and interior cells create accepted q_D and r_D footprints. The result is no longer a stable pore gas: the slit becomes a line source with total raw response $B_0(1, n) = 7n$ for $n \geq 2$.

14 Square, rectangle, and line-hole laws

14.1 Square and rectangle punctures

Let $D_{a,b} \subset T_L$ be an $a \times b$ rectangular hole in a stable active sheet. Assume $a, b \geq 2$ and the rectangle is not interacting with itself through the periodic boundary.

Theorem 14.1 (Rectangle puncture law). *For $a, b \geq 2$, the first-step raw birth count of an $a \times b$ rectangular puncture is*

$$B_0(a, b) = 4a + 4b + 12.$$

There are zero first-step deaths. Equivalently, if $P = 2a + 2b$ is the lattice perimeter, then

$$B_0(a, b) = 2P + 12.$$

Proof sketch. The planar selector gives exactly four in-plane births at the four defect corners. In each adjacent normal layer, the closed footprint band $2 \leq r_D \leq 4$ has $2a + 2b + 4$ sites. Thus

$$B_0(a, b) = 4 + 2(2a + 2b + 4) = 4a + 4b + 12.$$

For cells remaining in the sheet, the missing-neighbor count around a rectangle never forces $q_D \geq 4$ outside the defect mask in the first update, so deaths are zero. $\square$

Corollary 14.1 (Square puncture law). *For a square hole of side length $h \geq 2$,*

$$B_0(h, h) = 8h + 12,$$

with zero first-step deaths.

Corollary 14.2 (Overcompensation window). *For an $a \times b$ rectangle, the first update has more active cells than the intact sheet precisely when*

$$|A_1| > L^2 \iff (a - 4)(b - 4) < 28.$$

Equality gives exact compensation, and the reverse inequality gives net loss.

Proof. The initial active count is $|A_0| = L^2 - ab$. Since deaths are zero and births are $4a + 4b + 12$,

$$|A_1| - L^2 = -ab + 4a + 4b + 12 = 28 - (a - 4)(b - 4).$$

The sign of this expression gives the stated criterion. $\square$

15 Expanded rectangle morphology

Rectangles are the closed-form family that makes the defect calculus easy to audit. For an $a \times b$ rectangular hole with $a, b \geq 2$, the first-step response has three components.

15.1 Plane contribution

Only the four missing in-plane corner cells have $1 \leq q_D \leq 3$. Edge-but-not-corner defect cells have too many missing neighbors, and interior defect cells have still more. Therefore the in-plane rebirth contribution is exactly four.

15.2 Normal halo contribution

In each adjacent normal layer, the accepted footprint band $2 \leq r_D \leq 4$ forms a one-cell halo around the rectangle. The accepted count in one normal layer is

$$2(a + b) + 4.$$

There are two normal layers, so the normal contribution is

$$2\{2(a + b) + 4\} = 4(a + b) + 8.$$

Adding the four in-plane corner births gives

$$B_0(a, b) = 4(a + b) + 12.$$

15.3 Channel morphology

The same footprint classification splits the response into channels:

$$N_5 = 12, \quad N_6 = 4a + 4b - 16, \quad N_7 = 16.$$

The N_5 term is a constant corner/near-corner charge. The N_6 term is the length-dependent edge band. The N_7 term is a constant outer shallow halo. The default prime-corrected response removes the raw N_7 group, giving

$$B_\phi(a, b) = N_5 + N_6 = 4a + 4b - 4.$$

15.4 Overcompensation

The intact sheet contains L^2 active sites. Removing an $a \times b$ hole leaves $L^2 - ab$ active sites. The first step has no deaths for the rectangle family, so

$$|A_1| = L^2 - ab + B_0(a, b).$$

The first step overcompensates the intact sheet iff $|A_1| > L^2$, equivalently

$$4a + 4b + 12 > ab.$$

Rearranging gives

$$(a - 4)(b - 4) < 28.$$

This is a useful release-level result because it converts the event count into a clean geometry inequality.

15.5 Line-hole edge case

The one-cell-wide rectangle family is degenerate and should not be forced into the $a, b \geq 2$ rectangle law.

Theorem 15.1 (Line-hole edge law). *For a one-cell-wide line hole $1 \times n$, the raw and default prime-corrected first-step birth counts are*

$$B_0(1) = B_\phi(1) = 0,$$

and, for $n \geq 2$,

$$B_0(1, n) = 7n, \quad B_\phi(1, n) = 7n - 14.$$

Proof sketch. A single missing cell is a stable pinhole and produces no births. For $n \geq 2$, direct application of the q_D/r_D selectors to the one-cell-wide mask gives the raw line-source count $7n$. The default prime correction removes the two endpoint-related raw $S = 7$ normal-edge groups, totaling 14 births, yielding $7n - 14$. $\square$

16 Model tags: raw PM_0 and prime-sharpened PM_ϕ

Definition 16.1 (Raw model). PM_0 denotes the raw Poole cellular automaton using the integer Moore count $S = N_i(x)$. Dead cells are born for $S \in \{5, 6, 7\}$, and live cells survive for $S \in \{5, 6, 7, 8, 9\}$.

Definition 16.2 (Default prime-sharpened model). PM_ϕ denotes the default prime-sharpened model with

$$\Phi(S) = S + \sum_{p \in \{2,3,5,7,11,13,17,19,23\}} 0.35 \exp\left(-\frac{(S-p)^2}{0.01}\right),$$

and the same birth and survival windows applied to $\Phi(S)$. For integer channels relevant to the planar defect laws, the default PM_ϕ birth channels are $S = 5, 6$, while the survival channels remain $S = 5, 6, 7, 8, 9$.

Thus PM_ϕ is not a new theorem about the raw rule; it is a model-tagged variant used to study channel sensitivity, especially the effect of removing the raw $S = 7$ birth channel.

17 Rectangle channel spectrum and prime correction

Theorem 17.1 (Rectangle birth-channel spectrum). For $a, b \geq 2$, the raw first-step rectangle birth count decomposes by neighbor channel as

$$N_{S=5} = 12, \quad N_{S=6} = 4a + 4b - 16, \quad N_{S=7} = 16.$$

Therefore

$$B_0(a, b) = N_{S=5} + N_{S=6} + N_{S=7} = 4a + 4b + 12.$$

Proof sketch. The $S = 5$ events consist of the four in-plane corner births and eight normal-layer corner-footprint births. The $S = 7$ events consist of sixteen normal-layer edge-end events. The remaining normal-layer boundary band contributes $4a + 4b - 16$ events at $S = 6$. Summing these disjoint channel classes gives the rectangle law. $\square$

Corollary 17.1 (Default prime-corrected rectangle law). For $a, b \geq 2$, default PM_ϕ removes the sixteen raw $S = 7$ birth events and keeps the $S = 5$ and $S = 6$ birth events. Hence

$$B_\phi(a, b) = 12 + (4a + 4b - 16) = 4a + 4b - 4.$$

For a square hole,

$$B_\phi(h, h) = 8h - 4.$$

18 Irreversible work laws for rectangle defects

Define the per-event reconstruction information weight

$$I_k = \log 2 + \log \binom{26}{k}.$$

For rectangle holes, first-step deaths are zero, so the first-step irreversible event work is a channel-weighted birth work.

Theorem 18.1 (Raw rectangle irreversible work). For PM_0 and $a, b \geq 2$,

$$W_0(a, b) = 12I_5 + (4a + 4b - 16)I_6 + 16I_7.$$

Proof. Multiply the channel spectrum $(12, 4a + 4b - 16, 16)$ by the channel weights (I_5, I_6, I_7) and sum. $\square$

Theorem 18.2 (Prime rectangle irreversible work). *For default PM_ϕ and $a, b \geq 2$,*

$$W_\phi(a, b) = 12I_5 + (4a + 4b - 16)I_6.$$

The prime correction removes exactly $16I_7$ from the raw rectangle response.

Numerically,

$$I_5 \approx 11.7872, \quad I_6 \approx 13.0400, \quad I_7 \approx 14.0898,$$

so the removed work is approximately

$$16I_7 \approx 225.4369$$

natural-log Poole information-work units.

19 Model-tagged arbitrary planar selector

The raw planar theorem can be upgraded into a model-tagged selector for PM_0 and PM_ϕ .

Definition 19.1 (Dilation and channel indicators). *Let*

$$\text{Dil}_1(D) = \{u \in T_L : r_D(u) > 0\}$$

be the closed one-step dilation of D . For a model $M \in \{\text{PM}_0, \text{PM}_\phi\}$, let $\chi_B^M(S)$ be its birth-channel indicator and $\chi_S^M(S)$ its survival-channel indicator.

Theorem 19.1 (Model-tagged planar defect selector). *For a finite planar defect mask D in an otherwise stable active sheet, the first-step birth count in model M is*

$$B_M(D) = \sum_{u \in D} \chi_B^M(8 - q_D(u)) + 2 \sum_{u \in \text{Dil}_1(D)} \chi_B^M(9 - r_D(u)),$$

and the first-step death count is

$$D_M(D) = \sum_{u \in \text{Dil}_1(D) \setminus D} [1 - \chi_S^M(8 - q_D(u))].$$

Proof. The first term in $B_M(D)$ counts in-plane births at defect cells, where the in-plane neighbor count is $8 - q_D(u)$. The second term counts normal-layer births above and below the sheet, where the normal footprint count is $9 - r_D(u)$, giving the factor of two. Active sheet cells outside D can die only in the one-step dilation of D , where their in-plane count is $8 - q_D(u)$. The death indicator is therefore $1 - \chi_S^M(8 - q_D(u))$. Outside $\text{Dil}_1(D)$, all local counts are those of the intact stable sheet. $\square$

This theorem is more general than the rectangle laws. Rectangles are clean because they form a zero-death closed-form subfamily. Arbitrary polyomino holes can produce deaths and must be handled by the selector rather than by a perimeter-only formula.

20 Q/P-style rectangle reconstruction boundary

The rectangle laws define a limited geometry-to-count transducer.

Proposition 20.1 (Raw rectangle reconstruction from area and response). *For $a, b \geq 2$, the pair consisting of area $A = ab$ and raw birth count*

$$B_0 = 4a + 4b + 12$$

recovers the unordered side lengths $\{a, b\}$.

Proof. From B_0 , recover the sum

$$a + b = \frac{B_0 - 12}{4}.$$

Together with the product $ab = A$, the unordered pair $\{a, b\}$ is determined as the two roots of

$$z^2 - (a + b)z + ab = 0.$$

□

Proposition 20.2 (Prime rectangle reconstruction from area and response). *For $a, b \geq 2$, the pair $A = ab$ and*

$$B_\phi = 4a + 4b - 4$$

recovers $\{a, b\}$ by the same sum-product argument.

Remark 20.1 (Boundary of the reconstruction claim). Birth count alone is not enough, because it only fixes $a + b$. Many rectangle pairs share the same sum. The reconstruction is therefore a restricted rectangle-prior theorem, not a general reconstruction theorem for arbitrary holes.

21 Reconstruction algorithm under a rectangle prior

The Q/P-style reconstruction boundary is intentionally narrow. It does not reconstruct arbitrary masks. It reconstructs rectangle side lengths, up to exchange, when the area and first-step response are known and the rectangle prior is assumed.

Proposition 21.1 (Raw rectangle inversion algorithm). *Let $A = ab$ and $B_0 = 4a + 4b + 12$ for an unknown rectangle with $a, b \geq 2$. Then*

$$s = a + b = \frac{B_0 - 12}{4}, \quad p = ab = A.$$

The side lengths are the two roots of

$$z^2 - sz + p = 0.$$

Thus

$$\{a, b\} = \left\{ \frac{s - \sqrt{s^2 - 4p}}{2}, \frac{s + \sqrt{s^2 - 4p}}{2} \right\},$$

provided the discriminant is a nonnegative square and the roots are valid integer side lengths.

Proposition 21.2 (Prime rectangle inversion algorithm). *Let $A = ab$ and $B_\phi = 4a + 4b - 4$. Then*

$$s = a + b = \frac{B_\phi + 4}{4}, \quad p = A,$$

and the same quadratic inversion recovers $\{a, b\}$ up to exchange.

Remark 21.1 (Why response alone is ambiguous). The response count fixes $a + b$, not ab . Many rectangles share a perimeter-like response. Area supplies the missing product. This is why the paper should describe the result as a two-measure transducer, not a one-number reconstruction theorem.

22 Verification and empirical ledgers

22.1 v7 perimeter audit

The v7 audit independently checked the exact formulas against full three-dimensional updates. The main deterministic families were:

Test family	Lattice	Cases	Checked identity
Flat monolayer sheet	$L = 48$	1	fixed point, no births/deaths
Square holes	$L = 64$	26	$B(h) = 8h + 12$, $h = 3, \dots, 28$
Rectangular holes	$L = 72$	484	$B(a, b) = 4a + 4b + 12$, $a, b = 3, \dots, 24$

The audit is not a substitute for proof. Its role is to protect the formulas against indexing, boundary, channel-count, and implementation errors.

22.2 v9 theorem-level closure

The v9 amendment closed the following discrete-model gaps:

ID	Result
D01	Complete one-cell active plane is a fixed point.
D02	Survival window selects one-cell sheets over dust, lines, slabs, and bulk.
D03	Square puncture law: $B_0(h, h) = 8h + 12$, zero deaths.
D04	One-step planar defect calculus with q_D and r_D .
D05	Rectangle hole law: $B_0(a, b) = 4a + 4b + 12$, zero deaths.
D06	Line-hole edge law: $B_0(1, n) = 7n$, $B_\phi(1, n) = 7n - 14$ for $n \geq 2$.
D07	Rectangle channel spectrum: $N_5 = 12$, $N_6 = 4a + 4b - 16$, $N_7 = 16$.
D08	Prime-corrected rectangle law: $B_\phi(a, b) = 4a + 4b - 4$.
D09–D10	Raw and prime irreversible-work laws.
D11	Model-tagged planar selector for arbitrary finite planar holes.
D12–D13	Rectangle reconstruction from area plus raw or prime response.

The uploaded v9 audit reported 841 rectangle cases, 80 line-hole cases, 720 arbitrary planar-hole selector rows, and 1225 Q/P rectangle inversion cases, with the stated matching checks returning true.

22.3 v10 empirical atlas

The v10 atlas adds empirical and computational observations rather than new closed theorems. Its purpose is to identify the next candidate laws.

Observation	Evidence	Status
Raw planar defects cascade into high churn	Mean raw tail event density ≈ 0.359308	observation
Prime filtering suppresses planar cascades	Mean prime tail event density ≈ 0.080062	observation
Planar relaxation is pulse-like	Curves rise and then decay; monotone exponential is inadequate	model clue
3D void response is size-sensitive	Correlations above 0.98 for removed active volume against response	observation
Peak response is near-linear to mildly superlinear	Peak W beta: cube 0.979, sphere 1.202, ellipsoid 1.191	empirical fit
Integrated response is more superlinear	Integrated excess work scales more strongly than peak response	empirical fit

The strongest reported correlations were removed-active versus peak excess work 0.9861 and removed-active versus integrated excess work 0.9880. These are strong reproducible signals, but they should not be cited as proofs of a closed multi-step relaxation law or a closed non-planar void law.

23 Mean-field failure and correlation dependence

A naive independent-site mean-field approximation takes each site active with probability p_t . Then the neighbor count is approximated by

$$N \sim \text{Bin}(26, p_t).$$

The mean-field density update is

$$p_{t+1}^{\text{mf}} = (1 - p_t) \sum_{k=5}^7 \binom{26}{k} p_t^k (1 - p_t)^{26-k} + p_t \sum_{k=5}^9 \binom{26}{k} p_t^k (1 - p_t)^{26-k}.$$

This closure ignores spatial correlations, sheet geometry, cavities, and the fact that the rule selects local two-dimensional structure. It is useful primarily because it fails in a diagnostic way: persistent active regimes are not explained by independent random occupancy. They require correlated structure.

24 Continuum expansion of the 26-neighbor sum

Let the lattice spacing be a and suppose the binary field is coarse-grained to a smooth density $\rho(x)$. Formally,

$$\sum_{\delta \in \mathcal{N}_{26}} \rho(x + a\delta) = 26\rho(x) + 9a^2 \Delta \rho(x) + O(a^4),$$

because the first-order terms cancel by symmetry and

$$\frac{1}{2} \sum_{\delta \in \mathcal{N}_{26}} \delta_j^2 = 9$$

for each coordinate direction. Thus the local count threshold becomes a density-window condition with a curvature correction:

$$5 \lesssim 26\rho + 9a^2\Delta\rho \lesssim 7 + 2x.$$

This is not yet a physical continuum limit. It is a controlled expansion of the local counting operator. A true continuum theory would need scaling assumptions for x , ρ , defect density, correlation length, time step, and any physical calibration.

25 Rule-space interpretation

The nearby rules are best organized by two hysteresis choices: the lower birth threshold and the upper survival threshold. In this view:

- $B5-7/S5-9$ is the raw base rule with shallow-defect plasticity and sheet memory.
- $B5-7/S5-7$ removes upper memory and kills stable sheets.
- $B6-7/S5-9$ preserves upper sheet memory while suppressing the weakest $B5$ birth channel, making it a natural low-strain candidate.
- PM_ϕ preserves the same survival behavior in the relevant channels while removing raw $S = 7$ births under the default prime-sharpening parameters.

The rule-space lesson is that the upper survival band 8, 9 is the sheet-memory mechanism, while the upper birth edge B_7 is the plasticity mechanism.

26 Limitations and open problems

The following remain open and should be treated as future work:

1. a closed multi-step relaxation law after the first update;
2. a closed non-planar three-dimensional void law;
3. a full rule-space uniqueness theorem;
4. a rigorous correlation theory explaining stable active regimes beyond mean field;
5. topology-aware metrics for defect healing, freezing, and halo extrusion;
6. continuum calibration with an explicit scaling limit;
7. a disciplined bridge from defect work to any OTG, Q/P, gauge-force, or cosmology claims.

27 Conclusion

The Poole Defect Calculus is now a coherent discrete mathematical object. Its core equation

$$x_{t+1}(i) = \mathbf{1}\{5 \leq N_i(x_t) \leq 7 + 2x_t(i)\}$$

combines birth, survival, and hysteretic memory into one local threshold law. That law selects one-cell sheets, rejects dust and bulk, preserves stable active planes, freezes isolated pinholes, turns cracks into line sources, and maps finite planar defects to exact q_D/r_D selector counts. Rectangular defects are the cleanest closed-form family: they have zero first-step deaths, a perimeter-plus-corner birth law, an exact channel spectrum, a prime-corrected response, exact first-step work formulas, and a limited area-plus-response reconstruction theorem.

The empirical v10 atlas extends the program beyond first-step planar laws by measuring multi-step relaxation and non-planar void response. Its numbers are promising, especially the suppression of raw high-churn cascades by prime filtering and the strong removed-active-volume correlations in void response. But the correct claim boundary remains: first-step planar defect response is theorem-level; multi-step and 3D void laws are reproducible empirical targets awaiting closure.

Appendices

A.1 Minimal implementation requirements

A correct base implementation should:

1. use a three-dimensional Boolean array on a periodic cubic lattice;
2. compute the 26-neighbor Moore count by summing all shifts in $\{-1, 0, 1\}^3 \setminus \{0\}$;
3. apply birth to inactive cells with counts 5, 6, 7;
4. apply survival to active cells with counts 5, 6, 7, 8, 9;
5. report births, deaths, active counts, channel histograms, and event work separately;
6. keep theorem-level PM0 results separate from PM_ϕ variant results;
7. mark multi-step relaxation and 3D void results as empirical unless a proof is supplied.

A.2 Pseudocode

for each step:

```

S = 26_neighbor_count(A)
birth  = (~A) & (S >= 5) & (S <= 7)
survive = ( A) & (S >= 5) & (S <= 9)
death  = A & (~survive)
A_next = birth | survive

```

For default PM_ϕ , replace S by $\Phi(S)$ in the birth/survival inequalities while still recording raw integer channels for diagnostics.

A.3 Theorem ledger

ID	Name	Statement
D01	Stable plane	A complete one-cell active plane is a fixed point.
D02	Surface selector	The survival window preserves one-cell sheets and rejects dust, lines, slabs, and bulk.
D03	Planar selector	A defective sheet updates by the exact q_D/r_D calculus.
D04	Stable pinholes	Isolated one-cell pores are fixed.
D05	Square law	$B_0(h, h) = 8h + 12$, zero deaths.
D06	Rectangle law	$B_0(a, b) = 4a + 4b + 12 = 2P + 12$, zero deaths.
D07	Overcompensation	$ A_1 > L^2$ iff $(a - 4)(b - 4) < 28$.
D08	Line law	$B_0(1, n) = 7n$, $B_\phi(1, n) = 7n - 14$ for $n \geq 2$.
D09	Channel spectrum	$(N_5, N_6, N_7) = (12, 4a + 4b - 16, 16)$.
D10	Prime rectangle	$B_\phi(a, b) = 4a + 4b - 4$.
D11	Work laws	$W_0 = 12I_5 + (4a + 4b - 16)I_6 + 16I_7$; $W_\phi = 12I_5 + (4a + 4b - 16)I_6$.

ID	Name	Statement
D12	Model-tagged selector	Arbitrary planar masks have model-tagged $B_M(D)$ and $D_M(D)$ formulas.
D13	Q/P boundary	Area plus raw or prime rectangle response recovers $\{a, b\}$; response alone is ambiguous.

A.4 Complete theorem and claim-status ledger

The table below is the current release ledger. The status column is deliberately conservative. “Exact” means the statement follows from the local rule and finite combinatorics. “Verified” means the finite scripts matched the formula in the tested domain. “Empirical” means the item is reproducible numerical evidence but not a closed theorem.

ID	Type	Name	Statement	Status
D01	Theorem	Stable active plane	A complete one-cell coordinate plane is a fixed point of the base rule and of the default prime-tagged model in the relevant channel range.	Exact
D02	Theorem	Surface selector	Local survival preserves the two-dimensional coordinate-flat pattern and rejects local point, line, and filled-block patterns.	Exact
D03	Proposition	Square puncture	For square hole side $h \geq 2$, $B_0(h, h) = 8h + 12$ and first-step deaths are zero.	Exact/verified
D04	Theorem	Planar defect calculus	The first update of a defective sheet is determined by q_D and r_D .	Exact
D05	Corollary	Rectangle hole law	For $a, b \geq 2$, $B_0(a, b) = 4a + 4b + 12 = 2P + 12$ and first-step deaths are zero.	Exact/verified
D06	Theorem	Line-hole edge law	$B_0(1) = 0$, $B_0(1, n) = 7n$, $B_\phi(1) = 0$, and $B_\phi(1, n) = 7n - 14$ for $n \geq 2$.	Exact/verified
D07	Theorem	Channel spectrum	Rectangular births split as $N_5 = 12$, $N_6 = 4a + 4b - 16$, $N_7 = 16$.	Exact/verified
D08	Theorem	Prime rectangle law	Default PM_ϕ removes raw $S = 7$ rectangle births, giving $B_\phi(a, b) = 4a + 4b - 4$.	Exact in model
D09	Theorem	Raw work law	$W_0(a, b) = 12I_5 + (4a + 4b - 16)I_6 + 16I_7$.	Exact in model
D10	Theorem	Prime work law	$W_\phi(a, b) = 12I_5 + (4a + 4b - 16)I_6$.	Exact in model
D11	Theorem	Model-tagged selector	Arbitrary finite planar holes have exact $B_M(D)$ and $D_M(D)$ formulas using χ_B^M and χ_S^M .	Exact/verified
D12	Proposition	Raw rectangle reconstruction	Under a rectangle prior, $A = ab$ and B_0 recover $\{a, b\}$ up to exchange.	Exact/verified
D13	Proposition	Prime rectangle reconstruction	Under a rectangle prior, $A = ab$ and B_ϕ recover $\{a, b\}$ up to exchange.	Exact/verified
E01	Observation	Raw planar cascade	Multi-step raw planar defects enter a high-churn bath in the v10 run.	Empirical
E02	Observation	Prime suppression	Default PM_ϕ suppresses planar defect cascades relative to raw PM_0 .	Empirical
E03	Observation	Pulse-like relaxation	Planar relaxation curves rise and then decay; monotone exponential relaxation is inadequate.	Empirical/model clue

ID	Type	Name	Statement	Status
E04	Observation	3D void response	Peak and integrated void response correlate strongly with removed active volume in the tested range.	Empirical

A.5 Model dictionary

Definition A.5.1 (Base raw model PM_0). *The raw model PM_0 is the deterministic binary cellular automaton*

$$x_{t+1}(i) = \mathbf{1}\{5 \leq N_i(x_t) \leq 7 + 2x_t(i)\}.$$

It accepts dead-cell birth channels $S = 5, 6, 7$ and active-cell survival channels $S = 5, 6, 7, 8, 9$.

Definition A.5.2 (Default prime-sharpened model PM_ϕ). *The default prime-sharpened model applies the same birth and survival windows to*

$$\Phi(S) = S + \sum_{p \in \{2, 3, 5, 7, 11, 13, 17, 19, 23\}} 0.35 \exp\left(-\frac{(S-p)^2}{0.01}\right).$$

For the integer channels used in the planar defect laws, this keeps raw birth channels 5, 6 and rejects raw birth channel 7 because $\Phi(7) = 7.35 > 7$. The relevant survival channels remain 5, 6, 7, 8, 9.

Definition A.5.3 (Memory-reduced ablation). *The rule $B5-7/S5-7$ is the ablation that removes the upper survival memory band 8, 9. It kills a flat active sheet in one step and therefore fails the stable-sheet theorem.*

Definition A.5.4 (Low-strain candidate). *The rule $B6-7/S5-9$ preserves the upper sheet-memory band while suppressing the weakest $B5$ birth channel. It is not a replacement theorem; it is a rule-space candidate motivated by the surface-selector structure and earlier diagnostics.*

A.6 Defect operator cookbook

Let $D \subset T_L$ be a finite defect mask in an otherwise complete active coordinate sheet. All first-step planar-defect formulas are computed from two two-dimensional convolution counts:

$$q_D(u) = \sum_{\eta \in \{-1, 0, 1\}^2 \setminus \{0\}} \mathbf{1}\{u + \eta \in D\}, \quad r_D(u) = \sum_{\eta \in \{-1, 0, 1\}^2} \mathbf{1}\{u + \eta \in D\}.$$

The first is an open-neighborhood missing-count. The second is a closed 3×3 footprint missing-count.

Cell class	Count before update	Base-rule criterion
Active sheet cell $u \notin D$	$N(u, c) = 8 - q_D(u)$	Survives iff $q_D(u) \leq 3$
Inactive defect cell $u \in D$	$N(u, c) = 8 - q_D(u)$	Born in-plane iff $1 \leq q_D(u) \leq 3$
Inactive normal cell $(u, c \pm 1)$	$N(u, c \pm 1) = 9 - r_D(u)$	Born in normal halo iff $2 \leq r_D(u) \leq 4$
Layer $ z - c \geq 2$	$N = 0$ at first step	Remains inactive

The defect calculus is therefore local, finite, and computable by two planar convolutions. It is not a metaphorical derivative; it is a finite-difference response operator mapping a missing-cell mask D to births, deaths, and channel counts after one update.

A.6.1 General nearby-rule selector

For a nearby rule $Ba-b/Sc-d$, the same defective sheet has the first-step criteria

$$\begin{aligned} \text{active sheet survival} &\iff 8 - d \leq q_D(u) \leq 8 - c, \\ \text{in-plane defect birth} &\iff 8 - b \leq q_D(u) \leq 8 - a, \\ \text{normal-layer birth} &\iff 9 - b \leq r_D(u) \leq 9 - a. \end{aligned}$$

This compact formula makes the role of the +2 upper survival allowance transparent: widening d to 9 allows $q_D = 0$ sheet interiors to survive with count 8.

A.7 Detailed rectangle morphology and channel count

For $a, b \geq 2$, a rectangle has a defect interior, a four-corner set, side bands, and two adjacent normal layers. The raw birth law can be decomposed by both geometric location and neighbor channel.

Event class	Count	Channel	Explanation
In-plane defect corners	4	$S = 5$	A missing corner cell has three missing open-neighbor positions, so $8 - q_D = 5$.
In-plane non-corner defect sites	0 births	not eligible	Edge/interior missing cells have counts outside the birth band under the rectangle geometry.
Normal corner-footprint births	8	$S = 5$	Four corner footprints appear in both adjacent normal layers.
Normal edge-band births	$4a + 4b - 16$	$S = 6$	The regular side band in the two normal layers contributes the perimeter-linear term.
Normal edge-end births	16	$S = 7$	Two layers times eight edge-end positions produce the plasticity channel.
Deaths in sheet	0	none	Active sheet cells outside the rectangle never have $q_D \geq 4$ in the first update.

Summing the birth classes gives

$$B_0(a, b) = 12 + (4a + 4b - 16) + 16 = 4a + 4b + 12.$$

The prime-sharpened model removes precisely the $S = 7$ row, giving

$$B_\phi(a, b) = 12 + (4a + 4b - 16) = 4a + 4b - 4.$$

A.7.1 Work values

With

$$I_k = \log 2 + \log \binom{26}{k},$$

the channel values used in the v9/v10 paper are

$$I_5 \approx 11.7872183003, \quad I_6 \approx 13.0399812688, \quad I_7 \approx 14.0898033933.$$

Thus default prime sharpening removes the constant rectangle work

$$16I_7 \approx 225.4368542928$$

from the raw first-step rectangle response.

A.8 Rectangle reconstruction boundary expanded

The rectangle transducer is not a general hole-reconstruction theorem. It is a rectangle-prior theorem. The response provides a perimeter-like sum, while the area supplies the missing product.

Model	Response equation	Reconstruction from area $A = ab$
Raw PM_0	$B_0 = 4a + 4b + 12$	$s = (B_0 - 12)/4 = a + b$, $\Delta = s^2 - 4A$
Prime PM_ϕ	$B_\phi = 4a + 4b - 4$	$s = (B_\phi + 4)/4 = a + b$, $\Delta = s^2 - 4A$
Both	roots of $z^2 - sz + A = 0$	$\{a, b\} = \{(s - \sqrt{\Delta})/2, (s + \sqrt{\Delta})/2\}$

Response alone fixes $a + b$ but not ab . Therefore several rectangles can share a response class. The v9 audit found response-only ambiguity classes, so the release statement must remain: *area plus response reconstructs rectangular dimensions up to exchange under a rectangle prior.*

A.9 Verification ledger

Audit item	Domain	Result	Purpose
Flat monolayer sheet	one sheet, periodic lattice	no births/deaths	verifies stable-plane theorem and no-thickening.
Square punctures	$h = 3, \dots, 28$	$B_0(h, h) = 8h + 12$	checks square law against full 3D update.
Rectangular holes, early audit	$a, b = 3, \dots, 24$	484 matches	checks $B_0(a, b) = 4a + 4b + 12$.
Rectangles, v9 full audit	841 cases	all matched true	extended full 3D verification and channel tracking.
Line holes	80 cases	all matched true	validates the $1 \times n$ edge-case law.

Audit item	Domain	Result	Purpose
Arbitrary planar-hole selector	720 rows	births/deaths matched	all validates model-tagged $B_M(D)$ and $D_M(D)$ formulas against full 3D updates.
Q/P rectangle inversion	1225 cases	raw and prime area-plus-response recovery all matched	validates reconstruction boundary under rectangle prior.
Birth-only inversion check	tested rectangle domain	93 ambiguous classes	validates the warning that response alone is incomplete.

The audits are not substitutes for proofs when a proof exists. Their role is to prevent implementation errors, indexing errors, periodic-boundary mistakes, channel misclassification, and mistaken claims about reconstruction.

A.10 Expanded v10 empirical atlas

The v10 atlas is included because it advances the next two gaps: multi-step planar relaxation after first-step laws and non-planar 3D void response on an annealed PM_ϕ vacuum. It does not close those gaps as theorems.

A.10.1 v10 audit configuration

Parameter	Value
Preset	standard
Grid size	$N = 64$
Planar response length	80 steps
Vacuum anneal length	120 steps
Void response length	70 steps
Void trials per size	3
Random planar polyomino trials	5
Planar cases	9
3D void cases	36

A.10.2 Planar multi-step relaxation summary

The first-step event densities are tiny compared with the later tail response:

$$E_0(\text{PM}_0) \approx 0.000376, \quad E_0(\text{PM}_\phi) \approx 0.000294.$$

After 80 steps, the v10 run reported

$$E_{\text{tail}}(\text{PM}_0) \approx 0.359308, \quad E_{\text{tail}}(\text{PM}_\phi) \approx 0.080062.$$

The safe interpretation is that raw planar defects can cascade into high churn, while prime filtering suppresses that cascade by a large factor. The relaxation curves rise and then decay, so a monotone exponential is not the right model class. A better candidate family is

$$E(t) = E_{\infty} + At^p \exp(-t/\tau),$$

with baseline-matched controls and bootstrap confidence intervals.

A.10.3 Mean 3D void response table

Shape	Size	Volume proxy	Removed active	Peak excess W	Integrated excess W
cube	3	343.0	126.7	0.00515503	0.0269896
cube	4	729.0	298.0	0.0122313	0.110344
cube	5	1331.0	525.3	0.0213321	0.2274
cube	6	2197.0	873.7	0.0339162	0.422282
ellipsoid	3	99.0	39.7	0.00116729	0.00243775
ellipsoid	4	189.0	72.3	0.00293362	0.00898938
ellipsoid	5	281.0	110.3	0.00517274	0.0245721
ellipsoid	6	661.0	262.0	0.0112632	0.0859388
sphere	3	123.0	50.3	0.00127405	0.00394072
sphere	4	257.0	97.3	0.00328013	0.0134696
sphere	5	515.0	204.0	0.00710508	0.055229
sphere	6	925.0	365.3	0.0143826	0.152506

Using shape-size means, the peak excess work scaling versus removed active count has exponents approximately

$$\beta_{\text{cube}} \approx 0.979, \quad \beta_{\text{sphere}} \approx 1.202, \quad \beta_{\text{ellipsoid}} \approx 1.191.$$

Integrated excess work is more superlinear, with removed-active exponents approximately

$$\beta_{\text{cube,int}} \approx 1.425, \quad \beta_{\text{sphere,int}} \approx 1.853, \quad \beta_{\text{ellipsoid,int}} \approx 1.891.$$

The strongest safe claim is correlation and scaling over the tested range, not a universal closed 3D void law.

A.11 Reference implementation appendix

The following code is a compact reference implementation. It is not optimized. Its purpose is audit clarity.

```
import math
import numpy as np

SHIFTS_26 = tuple(
    (dx, dy, dz)
    for dx in (-1, 0, 1)
    for dy in (-1, 0, 1)
    for dz in (-1, 0, 1)
    if not (dx == 0 and dy == 0 and dz == 0)
```

```

)

PRIMES = (2, 3, 5, 7, 11, 13, 17, 19, 23)
PHI_LUT = np.arange(27, dtype=float)
for p in PRIMES:
    PHI_LUT += 0.35 * np.exp(-((np.arange(27) - p) ** 2) / 0.01)

I_LUT = np.array([math.log(2) + math.log(math.comb(26, k)) for k in range(27)])

def n26(A):
    """Periodic_26-neighbor_count_for_a_3D_boolean_array."""
    Ai = A.astype(np.int16, copy=False)
    S = np.zeros_like(Ai, dtype=np.int16)
    for dx, dy, dz in SHIFTS_26:
        S += np.roll(np.roll(np.roll(Ai, dx, 0), dy, 1), dz, 2)
    return S

def poole_step(A, mode="raw"):
    """One_Poole_step_mode='raw'_gives_PM0;_mode='prime'_gives_PM_phi."""
    S = n26(A)
    X = S.astype(float) if mode == "raw" else PHI_LUT[S]
    birth = (~A) & (X >= 5) & (X <= 7)
    survive = A & (X >= 5) & (X <= 9)
    death = A & (~survive)
    A1 = birth | survive
    event = birth | death
    metrics = {
        "active0": int(A.sum()),
        "births": int(birth.sum()),
        "deaths": int(death.sum()),
        "active1": int(A1.sum()),
        "events": int(event.sum()),
        "W_step": float((event * I_LUT[S]).sum()),
    }
    for k in range(27):
        metrics[f"B{k}"] = int((birth & (S == k)).sum())
        metrics[f"D{k}"] = int((death & (S == k)).sum())
    return A1, metrics

def sheet_with_rect_hole(L, a, b, z=None, x0=None, y0=None):
    """Full_coordinate_sheet_with_an_a_by_b_rectangular_hole."""
    if z is None: z = L // 2
    if x0 is None: x0 = (L - a) // 2
    if y0 is None: y0 = (L - b) // 2
    A = np.zeros((L, L, L), dtype=bool)
    A[:, :, z] = True
    A[x0:x0+a, y0:y0+b, z] = False
    return A

def predict_rectangle_raw(a, b):
    if a == 1 and b == 1:
        return {"births": 0, "deaths": 0}
    if a == 1:
        return {"births": 7 * b, "deaths": 0}

```



```

    if b == 1:
        return {"births": 7 * a, "deaths": 0}
    return {
        "births": 4*a + 4*b + 12,
        "deaths": 0,
        "B5": 12,
        "B6": 4*a + 4*b - 16,
        "B7": 16,
        "prime_births": 4*a + 4*b - 4,
    }

def verify_rectangles(L=72, amin=2, amax=24, mode="raw"):
    rows = []
    for a in range(amin, amax + 1):
        for b in range(amin, amax + 1):
            A = sheet_with_rect_hole(L, a, b)
            _, m = poole_step(A, mode=mode)
            p = predict_rectangle_raw(a, b)
            expected = p["births"] if mode == "raw" else p["prime_births"]
            rows.append({
                "a": a, "b": b,
                "observed_births": m["births"],
                "expected_births": expected,
                "observed_deaths": m["deaths"],
                "match": (m["births"] == expected and m["deaths"] == 0),
            })
    return rows

if __name__ == "__main__":
    rows = verify_rectangles(L=72, amin=2, amax=10, mode="raw")
    assert all(r["match"] for r in rows)
    print("rectangle_smoke_test_passed", len(rows), "cases")

```

A.12 Two-dimensional defect selector reference implementation

```

import numpy as np

SHIFTS_8 = tuple(
    (dx, dy) for dx in (-1, 0, 1) for dy in (-1, 0, 1)
    if not (dx == 0 and dy == 0)
)
SHIFTS_9 = tuple((dx, dy) for dx in (-1, 0, 1) for dy in (-1, 0, 1))

def roll2(A, dx, dy):
    return np.roll(np.roll(A, dx, axis=0), dy, axis=1)

def q_r_counts(D):
    """Open_q_D_and_closed_r_D_missing-cell_counts_on_a_2D_torus."""
    Di = D.astype(np.int16, copy=False)
    q = np.zeros_like(Di, dtype=np.int16)
    r = np.zeros_like(Di, dtype=np.int16)
    for dx, dy in SHIFTS_8:

```

```

        q += roll2(Di, dx, dy)
    for dx, dy in SHIFTS_9:
        r += roll2(Di, dx, dy)
    return q, r

def selector_counts(D, model="raw"):
    """First-step planar-selector births/deaths for PM0 or PM_phi."""
    q, r = q_r_counts(D)
    dil = r > 0
    if model == "raw":
        birth_channels = {5, 6, 7}
    elif model == "prime":
        birth_channels = {5, 6}
    else:
        raise ValueError("model must be raw or prime")
    survive_channels = {5, 6, 7, 8, 9}

    inplane_S = 8 - q
    normal_S = 9 - r

    inplane_births = D & np.isin(inplane_S, list(birth_channels))
    normal_births_one_layer = dil & np.isin(normal_S, list(birth_channels))
    active_sheet_dies = (~D) & dil & (~np.isin(inplane_S, list(survive_channels)))

    return {
        "births": int(inplane_births.sum() + 2 * normal_births_one_layer.sum()),
        "deaths": int(active_sheet_dies.sum()),
        "inplane_births": int(inplane_births.sum()),
        "normal_births_total": int(2 * normal_births_one_layer.sum()),
    }

```

A.13 Mean-field closure and why it fails structurally

The independent-site mean-field closure is

$$p_{t+1}^{\text{mf}} = (1 - p_t) \sum_{k=5}^7 \binom{26}{k} p_t^k (1 - p_t)^{26-k} + p_t \sum_{k=5}^9 \binom{26}{k} p_t^k (1 - p_t)^{26-k}.$$

This equation is useful as a null model because it discards the exact geometry that makes sheets stable. A perfect sheet has density $p = 1/L$ in a periodic L^3 lattice if it is one layer thick, so an independent-site model at the same density would not reproduce the sheet's deterministic $N = 8$ local support. The difference is not a small correction; it is the core of the calculus. Persistent activity is correlation-supported, not mean-field-supported.

A.14 Continuum expansion caveat

The formal expansion

$$\sum_{\delta \in \mathcal{N}_{26}} \rho(x + a\delta) = 26\rho(x) + 9a^2\Delta\rho(x) + O(a^4)$$

turns the local count into a density-window condition with curvature correction,

$$5 \lesssim 26\rho + 9a^2\Delta\rho \lesssim 7 + 2x.$$

This is only an expansion of the counting operator. It is not a continuum physics derivation. A physical bridge would require a scaling limit for voxel size, time step, density, defect density, correlation length, calibration constants, and measurement channel.

A.15 Glossary

Term	Meaning
Poole rule	The base $B5-7/S5-9$ three-dimensional binary cellular automaton.
Hysteretic support	The active state widens the upper support threshold from 7 to 9.
Sheet memory	The $+2x_t$ survival allowance that lets active sheet interiors with $N = 8$ persist.
Plasticity gate	The B_7 birth channel, especially visible in rectangle responses and prime filtering.
Defect mask D	The set of missing sites removed from an otherwise complete active sheet.
q_D	Open eight-neighborhood count of missing defect sites.
r_D	Closed 3×3 footprint count of missing defect sites.
Normal halo	Births in the layers immediately above and below a defective sheet.
Pinhole gas	A Chebyshev-separated collection of one-cell pores that remains fixed.
Rectangle law	$B_0(a, b) = 4a + 4b + 12$ for $a, b \geq 2$.
Prime correction	Default PM_ϕ removal of raw $S = 7$ birth events.
Irreversible work	Channel-weighted event count using $I_k = \log 2 + \log \binom{26}{k}$.
Q/P boundary	The restricted result that area plus rectangle response recovers side lengths up to exchange.
Empirical atlas	Reproducible numerical map of multi-step and 3D void behavior, not yet a theorem.

A.16 Open theorem roadmap

The next mathematical targets are:

1. classify all stable finite planar defect masks beyond separated pinholes;
2. derive a closed formula for first-step response of arbitrary polyominoes by boundary graph invariants;
3. classify when arbitrary planar holes produce deaths;
4. prove or refute a multi-step relaxation law for planar masks;
5. build a topology-aware metric for hole closure, extrusion, and fragmentation;

6. derive a non-planar 3D void response theorem for restricted background states;
7. formalize the rule-space uniqueness question: which nearby rules preserve sheets, reject slabs, and produce controlled defect halos;
8. define the exact conditions under which a dynamic Q/P extension can read multi-step response without conflating it with first-step static capture;
9. replace the continuum expansion caveat with a genuine scaling theorem, if one exists.

A.17 Final boundary statement

The robust release claim is this:

The Poole Defect Calculus is an exact first-step finite-combinatorial theory for how a hysteretic surface-selecting three-dimensional cellular automaton responds to planar defects in stable one-cell sheets. Its strongest closed laws are the q_D/r_D selector, the stable pinhole result, the square and rectangle birth laws, the line-hole edge law, the rectangle channel spectrum, the prime-corrected rectangle response, the channel-weighted work laws, and the rectangle-prior area-plus-response reconstruction theorem. Multi-step planar relaxation and non-planar 3D void response are currently reproducible empirical atlas results, not closed laws.

This is the clean line between a publishable discrete mathematics paper and broader speculative applications.

A.18 Publication-form proof capsules

This appendix restates the central exact claims in proof-capsule form so that the final release can be audited theorem by theorem without searching through the narrative.

A.18.1 D01: stable active plane

For an active coordinate plane H_c , every active site has exactly eight active neighbors in its own planar 3×3 block. Since $8 \in \{5, 6, 7, 8, 9\}$, every active sheet site survives. A site immediately above or below the sheet sees a full 3×3 footprint and hence has count 9. Since $9 \notin \{5, 6, 7\}$ for inactive birth, no adjacent normal cell is born. Every other inactive site has count 0. Therefore the full one-cell sheet is fixed.

A.18.2 D02: surface selector

A local coordinate r -flat has 3^r sites in the centered Moore block and $3^r - 1$ active neighbors of the center. The sequence for $r = 0, 1, 2, 3$ is 0, 2, 8, 26. The active survival window $[5, 9]$ contains exactly 8. Thus the local rule preserves the two-dimensional sheet pattern and rejects the local point, line, and bulk patterns. This is the exact mathematical basis for calling the rule surface-selecting.

A.18.3 D03–D05: square and rectangle response

The rectangle law is the parent result. For $a, b \geq 2$, the in-plane births occur only at the four defect corners. The normal-layer response is controlled by $2 \leq r_D \leq 4$ over the closed 3×3 footprint. In one normal layer this eligible footprint band has $2a + 2b + 4$ sites. There are two normal layers, so normal births contribute $4a + 4b + 8$. Adding the four in-plane corner births gives $4a + 4b + 12$. For a square $a = b = h$, this reduces to $8h + 12$.

The zero-death claim follows from the same local count. Outside the removed rectangle, no surviving sheet cell has four or more missing planar neighbors on the first step; hence every remaining active sheet cell satisfies $q_D \leq 3$ and survives.

A.18.4 D06: line-hole edge law

The line-hole family is not the $a, b \geq 2$ rectangle law with $a = 1$ substituted. The defect footprint degenerates. A single missing cell is a stable pinhole. For a one-cell-wide line of length $n \geq 2$, the open and closed defect counts along the line generate a raw line-source response of $7n$. The default prime-tagged model removes fourteen raw $S = 7$ endpoint-related births, giving $7n - 14$. The first-step death count remains zero.

A.18.5 D07–D08: channel spectrum and prime correction

For a nondegenerate rectangle, the raw births split into three disjoint integer neighbor channels. There are twelve $S = 5$ births, $4a + 4b - 16$ $S = 6$ births, and sixteen $S = 7$ births. Default prime sharpening changes the evaluated count from S to $\Phi(S)$, and $\Phi(7) = 7.35 > 7$ pushes raw $S = 7$ births outside the birth window. Raw $S = 5$ and $S = 6$ events remain accepted. Therefore the default prime-corrected count is raw minus sixteen:

$$B_\phi(a, b) = B_0(a, b) - 16 = 4a + 4b - 4.$$

A.18.6 D09–D10: irreversible work

The work formula is channel bookkeeping. Since the rectangle family has zero first-step deaths, all first-step events are births. Weighting each birth channel $S = k$ by $I_k = \log 2 + \log \binom{26}{k}$ gives the raw formula

$$W_0 = 12I_5 + (4a + 4b - 16)I_6 + 16I_7.$$

Default prime correction removes the $S = 7$ group and hence removes $16I_7$ exactly.

A.18.7 D11: arbitrary planar selector

The model-tagged selector follows from the same four local cell classes. In-plane defect cells have count $8 - q_D$. Normal-layer cells have count $9 - r_D$. Active cells outside the defect can die only within the closed one-step dilation of D , because outside that dilation the local pattern is exactly the intact stable sheet. Replacing the raw birth and survival windows with model-specific indicators χ_B^M and χ_S^M gives the full model-tagged formulas.

A.18.8 D12–D13: rectangle reconstruction boundary

The response count gives a sum. The area gives a product. For raw rectangles,

$$s = a + b = \frac{B_0 - 12}{4}, \quad A = ab.$$

For default prime rectangles,

$$s = a + b = \frac{B_\phi + 4}{4}, \quad A = ab.$$

In either case, a and b are the two roots of $z^2 - sz + A = 0$. Without area, the response fixes only s , so response alone cannot reconstruct arbitrary rectangle side lengths.

A.19 Full reference verifier script

The following single-file verifier is intended as the starting point for the release repository. It covers the base 26-neighbor engine, the prime-tagged variant, planar q_D/r_D selectors, rectangle predictions, line predictions, smoke tests, and CSV outputs.

```
# poole_defect_calculus_reference_verify.py
# Reference verifier for Poole Defect Calculus archival release.
# This is written for clarity over speed.

import math
import json
import random
from pathlib import Path
import numpy as np
import pandas as pd

OUT = Path("poole_defect_reference_verify_out")
OUT.mkdir(exist_ok=True, parents=True)

SHIFTS_26 = tuple(
```

```

(dx, dy, dz)
for dx in (-1, 0, 1)
for dy in (-1, 0, 1)
for dz in (-1, 0, 1)
if not (dx == 0 and dy == 0 and dz == 0)
)
SHIFTS_8 = tuple(
(dx, dy)
for dx in (-1, 0, 1)
for dy in (-1, 0, 1)
if not (dx == 0 and dy == 0)
)
SHIFTS_9 = tuple((dx, dy) for dx in (-1, 0, 1) for dy in (-1, 0, 1))

PHI_LUT = np.arange(27, dtype=float)
for p in (2, 3, 5, 7, 11, 13, 17, 19, 23):
    PHI_LUT += 0.35 * np.exp(-(np.arange(27) - p) ** 2) / 0.01)

I_LUT = np.array([math.log(2) + math.log(math.comb(26, k)) for k in range(27)])

def roll3(A, dx, dy, dz):
    return np.roll(np.roll(np.roll(A, dx, axis=0), dy, axis=1), dz, axis=2)

def roll2(A, dx, dy):
    return np.roll(np.roll(A, dx, axis=0), dy, axis=1)

def n26(A):
    Ai = A.astype(np.int16, copy=False)
    S = np.zeros_like(Ai, dtype=np.int16)
    for dx, dy, dz in SHIFTS_26:
        S += roll3(Ai, dx, dy, dz)
    return S

def poole_step(A, mode="raw"):
    S = n26(A)
    if mode == "raw":
        X = S.astype(float)
    elif mode == "prime":
        X = PHI_LUT[S]
    else:
        raise ValueError("mode must be raw or prime")
    birth = (~A) & (X >= 5) & (X <= 7)
    survive = A & (X >= 5) & (X <= 9)
    death = A & (~survive)
    A1 = birth | survive
    event = birth | death
    row = {
        "mode": mode,
        "active0": int(A.sum()),
        "births": int(birth.sum()),
        "deaths": int(death.sum()),
        "active1": int(A1.sum()),
        "events": int(event.sum()),
        "W_step": float((event * I_LUT[S]).sum()),
    }

```

```

    }
    for k in range(27):
        row[f"B{k}"] = int((birth & (S == k)).sum())
        row[f"D{k}"] = int((death & (S == k)).sum())
    return A1, row

def full_sheet(L, z=None):
    if z is None:
        z = L // 2
    A = np.zeros((L, L, L), dtype=bool)
    A[:, :, z] = True
    return A

def sheet_with_mask(L, D, z=None):
    A = full_sheet(L, z=z)
    if z is None:
        z = L // 2
    for x, y in D:
        A[x % L, y % L, z] = False
    return A

def rect_mask(L, a, b, x0=None, y0=None):
    if x0 is None:
        x0 = (L - a) // 2
    if y0 is None:
        y0 = (L - b) // 2
    return {(x, y) for x in range(x0, x0 + a) for y in range(y0, y0 + b)}

def line_mask(L, n, orientation="x", x0=None, y0=None):
    if x0 is None:
        x0 = L // 2
    if y0 is None:
        y0 = L // 2
    if orientation == "x":
        return {(x0 + i) % L, y0) for i in range(n)}
    if orientation == "y":
        return {(x0, (y0 + i) % L) for i in range(n)}
    raise ValueError("orientation must be x or y")

def q_r_counts_from_mask(L, D):
    M = np.zeros((L, L), dtype=bool)
    for x, y in D:
        M[x % L, y % L] = True
    Mi = M.astype(np.int16, copy=False)
    q = np.zeros_like(Mi, dtype=np.int16)
    r = np.zeros_like(Mi, dtype=np.int16)
    for dx, dy in SHIFTS_8:
        q += roll2(Mi, dx, dy)
    for dx, dy in SHIFTS_9:
        r += roll2(Mi, dx, dy)
    return M, q, r

def selector_predict(L, D, mode="raw"):
    M, q, r = q_r_counts_from_mask(L, D)

```



```

dil = r > 0
if mode == "raw":
    birth_channels = {5, 6, 7}
elif mode == "prime":
    birth_channels = {5, 6}
else:
    raise ValueError("mode must be raw or prime")
survive_channels = {5, 6, 7, 8, 9}
inplane_S = 8 - q
normal_S = 9 - r
inplane_births = M & np.isin(inplane_S, list(birth_channels))
normal_births = dil & np.isin(normal_S, list(birth_channels))
deaths = (~M) & dil & (~np.isin(inplane_S, list(survive_channels)))
return {
    "selector_births": int(inplane_births.sum() + 2 * normal_births.sum()),
    "selector_deaths": int(deaths.sum()),
}

def predict_rect(a, b, mode="raw"):
    if a == 1 and b == 1:
        raw = 0
        prime = 0
    elif a == 1:
        raw = 7 * b
        prime = 7 * b - 14 if b >= 2 else 0
    elif b == 1:
        raw = 7 * a
        prime = 7 * a - 14 if a >= 2 else 0
    else:
        raw = 4 * a + 4 * b + 12
        prime = 4 * a + 4 * b - 4
    return raw if mode == "raw" else prime

def verify_stable_sheet(L=32):
    A = full_sheet(L)
    _, m = poole_step(A, mode="raw")
    return {"test": "stable_sheet", "L": L, "births": m["births"], "deaths": m["deaths"],
        "match": m["births"] == 0 and m["deaths"] == 0}

def verify_rectangles(L=72, amin=2, amax=24, mode="raw"):
    rows = []
    for a in range(amin, amax + 1):
        for b in range(amin, amax + 1):
            D = rect_mask(L, a, b)
            A = sheet_with_mask(L, D)
            _, m = poole_step(A, mode=mode)
            sel = selector_predict(L, D, mode=mode)
            pred = predict_rect(a, b, mode=mode)
            rows.append({
                "family": "rectangle",
                "mode": mode,
                "a": a,
                "b": b,
                "observed_births": m["births"],

```

```

        "predicted_births": pred,
        "selector_births": sel["selector_births"],
        "observed_deaths": m["deaths"],
        "selector_deaths": sel["selector_deaths"],
        "match": m["births"] == pred and m["deaths"] == 0 and sel["
selector_births"] == m["births"] and sel["selector_deaths"] == m["deaths"],
    })
    return rows

def verify_lines(L=72, nmax=40, mode="raw"):
    rows = []
    for orientation in ("x", "y"):
        for n in range(1, nmax + 1):
            D = line_mask(L, n, orientation=orientation)
            A = sheet_with_mask(L, D)
            _, m = poole_step(A, mode=mode)
            sel = selector_predict(L, D, mode=mode)
            pred = predict_rect(1, n, mode=mode)
            rows.append({
                "family": "line",
                "mode": mode,
                "orientation": orientation,
                "n": n,
                "observed_births": m["births"],
                "predicted_births": pred,
                "selector_births": sel["selector_births"],
                "observed_deaths": m["deaths"],
                "selector_deaths": sel["selector_deaths"],
                "match": m["births"] == pred and m["deaths"] == 0 and sel["
selector_births"] == m["births"],
            })
    return rows

def random_polyomino(L, size, seed):
    rng = random.Random(seed)
    cells = {(L // 2, L // 2)}
    frontier = list(cells)
    while len(cells) < size:
        x, y = rng.choice(frontier)
        dx, dy = rng.choice([(1,0), (-1,0), (0,1), (0,-1)])
        c = ((x + dx) % L, (y + dy) % L)
        cells.add(c)
        frontier.append(c)
    return cells

def verify_random_polyominoes(L=64, sizes=(3,4,5,6,8,10), trials=10, mode="raw"):
    rows = []
    for size in sizes:
        for t in range(trials):
            D = random_polyomino(L, size, seed=1000 + 31*size + t)
            A = sheet_with_mask(L, D)
            _, m = poole_step(A, mode=mode)
            sel = selector_predict(L, D, mode=mode)
            rows.append({

```

```

        "family": "polyomino",
        "mode": mode,
        "size": size,
        "trial": t,
        "observed_births": m["births"],
        "selector_births": sel["selector_births"],
        "observed_deaths": m["deaths"],
        "selector_deaths": sel["selector_deaths"],
        "match": m["births"] == sel["selector_births"] and m["deaths"] == sel["
selector_deaths"],
    })
    return rows

def main():
    all_rows = []
    all_rows.append(verify_stable_sheet())
    for mode in ("raw", "prime"):
        all_rows.extend(verify_rectangles(mode=mode))
        all_rows.extend(verify_lines(mode=mode))
        all_rows.extend(verify_random_polyominoes(mode=mode))
    df = pd.DataFrame(all_rows)
    df.to_csv(OUT / "verification_rows.csv", index=False)
    summary = {
        "rows": int(len(df)),
        "all_match": bool(df["match"].all()),
        "failures": int((~df["match"]).sum()),
    }
    (OUT / "summary.json").write_text(json.dumps(summary, indent=2), encoding="utf-8")
    print(json.dumps(summary, indent=2))

if __name__ == "__main__":
    main()

```

A.20 Empirical atlas reference harness

The v10 atlas needs a separate harness because it measures multi-step behavior rather than theorem-level first-step laws.

```

# poole_defect_atlas_v10_reference.py
# Empirical atlas reference harness. This is not a theorem prover.

import json
import math
import time
from pathlib import Path
import numpy as np
import pandas as pd

# Reuse n26, poole_step, sheet_with_mask, rect_mask, line_mask, random_polyomino
# from the reference verifier or copy the definitions into this file.

PRESET = "standard"
if PRESET == "quick":

```

```

N = 48
PLANAR_T = 50
VACUUM_ANNEAL = 60
VOID_T = 40
VOID_TRIALS = 2
elif PRESET == "standard":
    N = 64
    PLANAR_T = 80
    VACUUM_ANNEAL = 120
    VOID_T = 70
    VOID_TRIALS = 3
else:
    N = 96
    PLANAR_T = 140
    VACUUM_ANNEAL = 240
    VOID_T = 120
    VOID_TRIALS = 6

OUT = Path("poole_defect_atlas_v10_reference_out")
OUT.mkdir(parents=True, exist_ok=True)

def run_steps(A, T, mode="prime", label="case", regime="run"):
    rows = []
    W_cum = 0.0
    for t in range(T):
        A, m = poole_step(A, mode=mode)
        W_cum += m["W_step"]
        m.update({
            "t": t,
            "mode": mode,
            "label": label,
            "regime": regime,
            "W_cum": W_cum,
            "W_cum_density": W_cum / A.size,
            "event_density": m["events"] / A.size,
            "W_step_density": m["W_step"] / A.size,
        })
        rows.append(m)
    return A, pd.DataFrame(rows)

def planar_cases(N):
    cases = []
    cases.append(("rect_4x4", rect_mask(N, 4, 4)))
    cases.append(("rect_6x10", rect_mask(N, 6, 10)))
    cases.append(("line_12", line_mask(N, 12, orientation="x")))
    cases.append(("line_20", line_mask(N, 20, orientation="y")))
    for size in (6, 10, 16, 24, 32):
        cases.append((f"poly_{size}", random_polyomino(N, size, seed=20260619 + size)))
    return cases

def run_planar_atlas():
    frames = []
    for label, D in planar_cases(N):
        A0 = sheet_with_mask(N, D)

```

```

        for mode in ("raw", "prime"):
            _, hist = run_steps(A0.copy(), PLANAR_T, mode=mode, label=label, regime="
planar")
            frames.append(hist)
        df = pd.concat(frames, ignore_index=True)
        df.to_csv(OUT / "planar_multistep_history.csv", index=False)
        return df

def make_random_vacuum(N, p=0.18, seed=0):
    rng = np.random.default_rng(seed)
    return rng.random((N, N, N)) < p

def anneal_vacuum(N, T, seed=0):
    A = make_random_vacuum(N, p=0.18, seed=seed)
    A, hist = run_steps(A, T, mode="prime", label="anneal", regime="anneal")
    return A, hist

def void_mask(N, shape, size):
    cx = cy = cz = N // 2
    X, Y, Z = np.ogrid[:N, :N, :N]
    if shape == "cube":
        r = size
        return (abs(X-cx) <= r) & (abs(Y-cy) <= r) & (abs(Z-cz) <= r)
    if shape == "sphere":
        r = size
        return ((X-cx)**2 + (Y-cy)**2 + (Z-cz)**2) <= r*r
    if shape == "ellipsoid":
        rx, ry, rz = size, max(1, size-1), size+1
        return ((X-cx)/rx)**2 + ((Y-cy)/ry)**2 + ((Z-cz)/rz)**2 <= 1
    raise ValueError("shape must be cube, sphere, or ellipsoid")

def run_void_atlas():
    summary_rows = []
    hist_frames = []
    for trial in range(VOID_TRIALS):
        baseline, anneal_hist = anneal_vacuum(N, VACUUM_ANNEAL, seed=8000+trial)
        baseline_final, base_hist = run_steps(baseline.copy(), VOID_T, mode="prime",
label="baseline", regime="void_control")
        for shape in ("cube", "sphere", "ellipsoid"):
            for size in (3, 4, 5, 6):
                mask = void_mask(N, shape, size)
                A = baseline.copy()
                removed_active = int((A & mask).sum())
                A[mask] = False
                label = f"{shape}_{size}_trial_{trial}"
                _, hist = run_steps(A, VOID_T, mode="prime", label=label, regime="void")
                merged = hist[["t", "W_step_density", "event_density"]].copy()
                merged["baseline_W_density"] = base_hist["W_step_density"].values
                merged["excess_W_density"] = merged["W_step_density"] - merged["
baseline_W_density"]
                merged["shape"] = shape
                merged["size"] = size
                merged["trial"] = trial
                hist_frames.append(merged)

```

```

        summary_rows.append({
            "shape": shape,
            "size": size,
            "trial": trial,
            "volume_proxy": int(mask.sum()),
            "removed_active": removed_active,
            "peak_excess_W_density": float(merged["excess_W_density"].max()),
            "integrated_excess_W_density": float(merged["excess_W_density"].sum())
        ),
    })
    void_summary = pd.DataFrame(summary_rows)
    void_hist = pd.concat(hist_frames, ignore_index=True)
    void_summary.to_csv(OUT / "void3d_summary.csv", index=False)
    void_hist.to_csv(OUT / "void3d_history.csv", index=False)
    return void_summary, void_hist

def fit_power_law(df, xcol, ycol):
    g = df[(df[xcol] > 0) & (df[ycol] > 0)].copy()
    X = np.log(g[xcol].values)
    Y = np.log(g[ycol].values)
    beta, logC = np.polyfit(X, Y, 1)
    pred = np.exp(logC) * g[xcol].values ** beta
    rmse = float(np.sqrt(np.mean((g[ycol].values - pred) ** 2)))
    return {"x": xcol, "y": ycol, "beta": float(beta), "C": float(np.exp(logC)), "rmse":
    rmse}

def summarize_atlas(planar_df, void_summary):
    tail = planar_df[planar_df["t"] >= planar_df["t"].max() - 9]
    tail_summary = tail.groupby("mode", as_index=False).agg(
        mean_tail_event_density=("event_density", "mean"),
        mean_tail_W_density=("W_step_density", "mean"),
        mean_final_W_cum_density=("W_cum_density", "mean"),
    )
    fit_rows = []
    for shape, g in void_summary.groupby("shape"):
        means = g.groupby("size", as_index=False).mean(numeric_only=True)
        for ycol in ("peak_excess_W_density", "integrated_excess_W_density"):
            fit = fit_power_law(means, "removed_active", ycol)
            fit["shape"] = shape
            fit_rows.append(fit)
    fits = pd.DataFrame(fit_rows)
    tail_summary.to_csv(OUT / "planar_tail_summary.csv", index=False)
    fits.to_csv(OUT / "void3d_scaling_fits.csv", index=False)
    summary = {
        "preset": PRESET,
        "N": N,
        "PLANAR_T": PLANAR_T,
        "VACUUM_ANNEAL": VACUUM_ANNEAL,
        "VOID_T": VOID_T,
        "VOID_TRIALS": VOID_TRIALS,
        "generated": time.ctime(),
        "claim_boundary": "empirical_atlas, not closed theorem",
    }
    (OUT / "summary.json").write_text(json.dumps(summary, indent=2), encoding="utf-8")

```

```

    return tail_summary, fits

def main():
    planar_df = run_planar_atlas()
    void_summary, void_hist = run_void_atlas()
    tail_summary, fits = summarize_atlas(planar_df, void_summary)
    print(tail_summary)
    print(fits)

# main() requires the reference verifier functions to be present.

```

A.21 CSV schema appendix

File	Required columns	Meaning
verification_rows.csv	family, mode, observed_births, predicted_births, observed_deaths, match	Theorem-level first-step verification rows.
planar_multistep_history.csv	family, mode, t, active, births, deaths, events, event_density, W_step_density, W_cum_density	Empirical planar relaxation histories.
void3d_summary.csv	shape, size, trial, volume_proxy, moved_active, peak_excess_W_density, integrated_excess_W_density	3D void response summary per case.
void3d_history.csv	t, shape, size, trial, W_step_density, baseline_W_density, excess_W_density	3D void response curves.
void3d_scaling_fits.csv	shape, x, y, beta, C, rmse	Candidate scaling fits for empirical law discovery.
summary.json	preset, N, step counts, cases, generated, claim_boundary	Machine-readable run record.

A.22 Claim boundaries and overclaim controls

A robust paper should explicitly avoid these claims unless future proof or calibration is supplied:

1. Do not say that the v10 atlas proves a closed multi-step relaxation law.
2. Do not say that the v10 void atlas proves a closed three-dimensional void law.

3. Do not say that the continuum expansion proves physical spacetime curvature.
4. Do not say that rectangle reconstruction reconstructs arbitrary masks.
5. Do not say that static first-step Q/P capture is the same as dynamic multi-step response capture.
6. Do not say that prime sharpening is a theorem about the raw rule; it is a model-tagged variant.
7. Do not say that audits replace proofs for exact laws; audits verify implementations and tested domains.

The strength of the Poole Defect Calculus is that its exact layer is already interesting without those overextensions.

Evidence and Verification Appendices

A.23 Evidence role

This appendix records regenerated evidence artifacts for the manuscript. It is not a substitute for the proofs in the main text. Its purpose is narrower: provide reproducible finite-lattice audits for the theorem-level first-step laws, record those audits as CSV/JSON/PNG files, and define a concrete reference verifier that other readers can run.

The The evidence appendix uses the raw Poole Manifold rule

$$x_{t+1}(i) = \mathbf{1}\{5 \leq N_i(x_t) \leq 7 + 2x_t(i)\}$$

on finite periodic cubic lattices. The default prime-corrected quantities in this appendix are model-tagged post-selector quantities: they reject raw birth-channel candidates with neighbor count seven. They are included because the theorem ledger already separates PM0 from PM_phi.

Evidence boundary. The rectangle, line-hole, channel, selector, and reconstruction audits are checks of the closed first-step theorem layer. The multi-step relaxation CSV is an empirical reference trace only. It is not a closed law and should not be described as one in this paper.

A.24 Pass/fail summary

All closed-layer checks passed in this regenerated evidence run. The run includes 841 rectangle cases, 80 line-hole cases, 720 random planar selector cases, and 1225 Q/P-style rectangle inversion cases. The empirical timeseries rows are included as reference traces and are excluded from theorem-level pass/fail language.

A.25 Rectangle, square, channel, prime, and overcompensation audit

The rectangle audit regenerates the closed first-step laws over $2 \leq a, b \leq 30$. This gives 841 cases. For every case the script constructs a complete stable active sheet, removes an $a \times b$ rectangle, applies the raw PM0 update once, and records births, deaths, active counts, and birth-channel counts.

Table 11: Regenerated evidence summary.

metric	value
rectangle_cases	841
rectangle_raw_matches	841
rectangle_channel_matches	841
rectangle_prime_matches	841
rectangle_overcomp_matches	841
line_cases	80
line_raw_matches	80
line_prime_matches	80
selector_random_cases	720
selector_random_matches	720
selector_random_nonlocal_events_total	0
qp_inversion_cases	1225
qp_raw_inversion_matches	1225
qp_prime_inversion_matches	1225
empirical_timeseries_rows	78
all_closed_layer_checks_passed	True

The checked laws are

$$B_0(a, b) = 4a + 4b + 12, \quad D_0(a, b) = 0,$$

$$(N_5, N_6, N_7) = (12, 4a + 4b - 16, 16),$$

$$B_\phi(a, b) = N_5 + N_6 = 4a + 4b - 4,$$

and the first-step overcompensation criterion

$$|A_1| > |A_{\text{sheet}}| \iff (a - 4)(b - 4) < 28.$$

Table 12: Representative rows from the rectangle/channel/prime audit.

a	b	B0	B0 pred	D	N5	N6	N7	Bphi	Bphi pred	over	ok
2	2	28	28	0	12	0	16	12	12	True	True
2	3	32	32	0	12	4	16	16	16	True	True
2	10	60	60	0	12	32	16	44	44	True	True
2	30	140	140	0	12	112	16	124	124	True	True
10	10	92	92	0	12	64	16	76	76	False	True
10	20	132	132	0	12	104	16	116	116	False	True
10	30	172	172	0	12	144	16	156	156	False	True
30	30	252	252	0	12	224	16	236	236	False	True

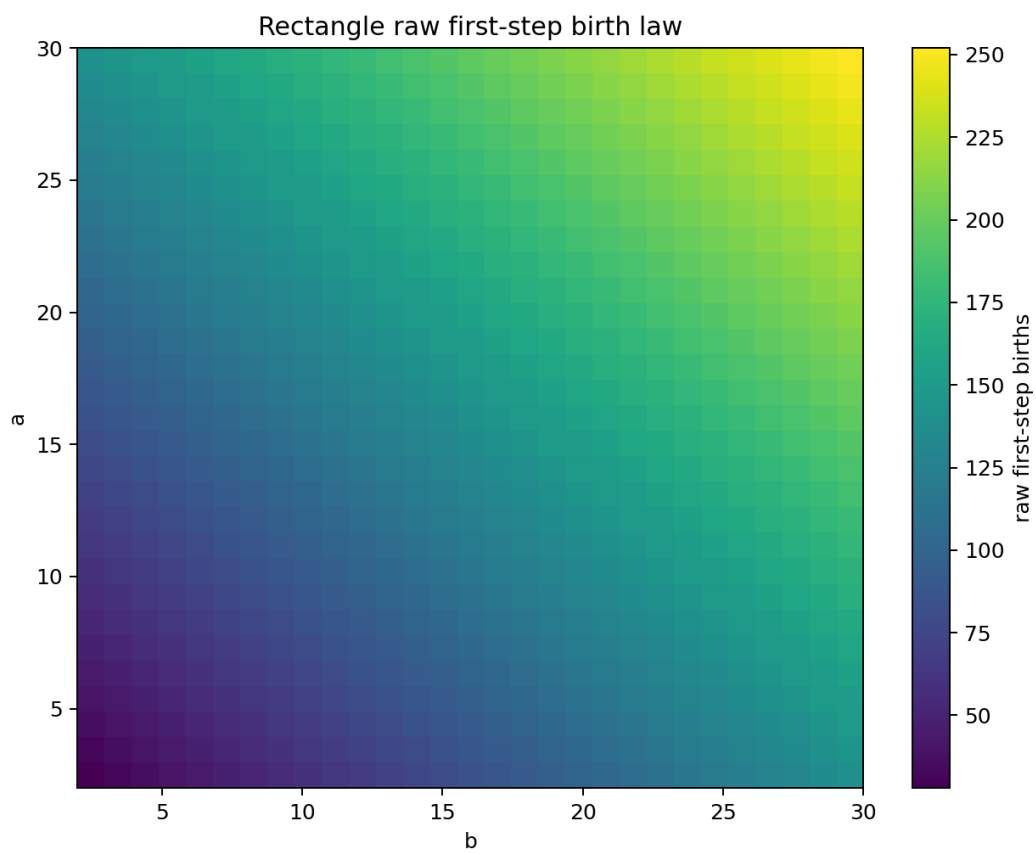


Figure 1: Regenerated rectangle raw first-step birth surface.

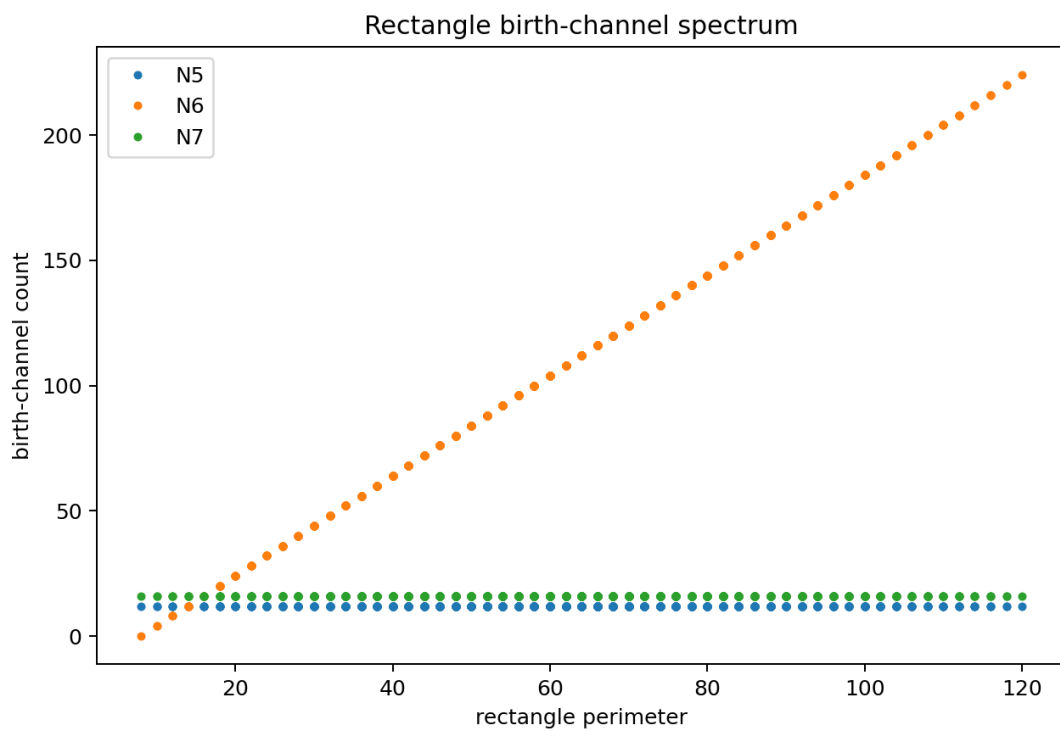


Figure 2: Regenerated rectangle birth-channel spectrum. The constant twelve N_5 and sixteen N_7 edge/corner channels separate from the perimeter-dependent N_6 channel.

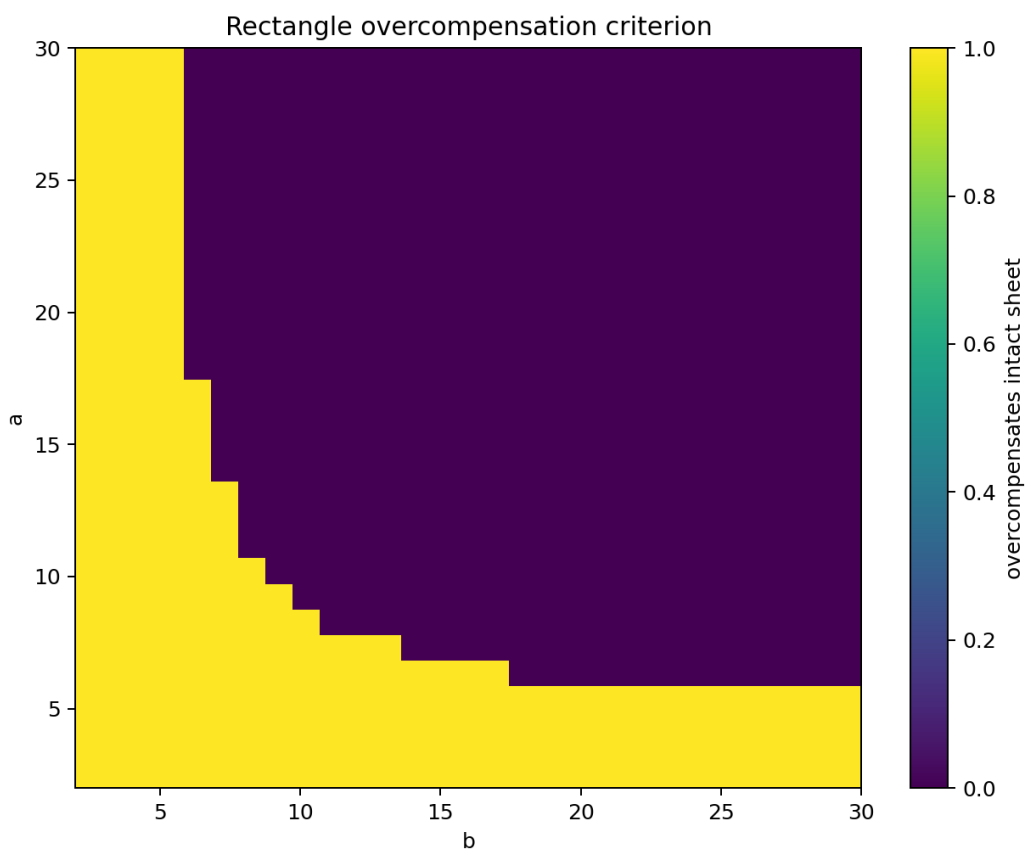


Figure 3: Regenerated first-step overcompensation map for rectangles over $2 \leq a, b \leq 30$.

A.26 Line-hole edge audit

The one-cell-wide family is a degenerate boundary case and is not folded into the $a, b \geq 2$ rectangle law. The regenerated audit covers $1 \leq n \leq 80$. It checks

$$B_0(1) = B_\phi(1) = 0,$$

and for $n \geq 2$,

$$B_0(1, n) = 7n, \quad B_\phi(1, n) = 7n - 14.$$

Table 13: Representative rows from the line-hole edge audit.

n	B0	B0 pred	D	N5	N6	N7	Bphi	Bphi pred	ok
1	0	0	0	0	0	0	0	0	True
2	14	14	0	0	0	14	0	0	True
3	21	21	0	0	7	14	7	7	True
10	70	70	0	0	56	14	56	56	True
40	280	280	0	0	266	14	266	266	True
80	560	560	0	0	546	14	546	546	True

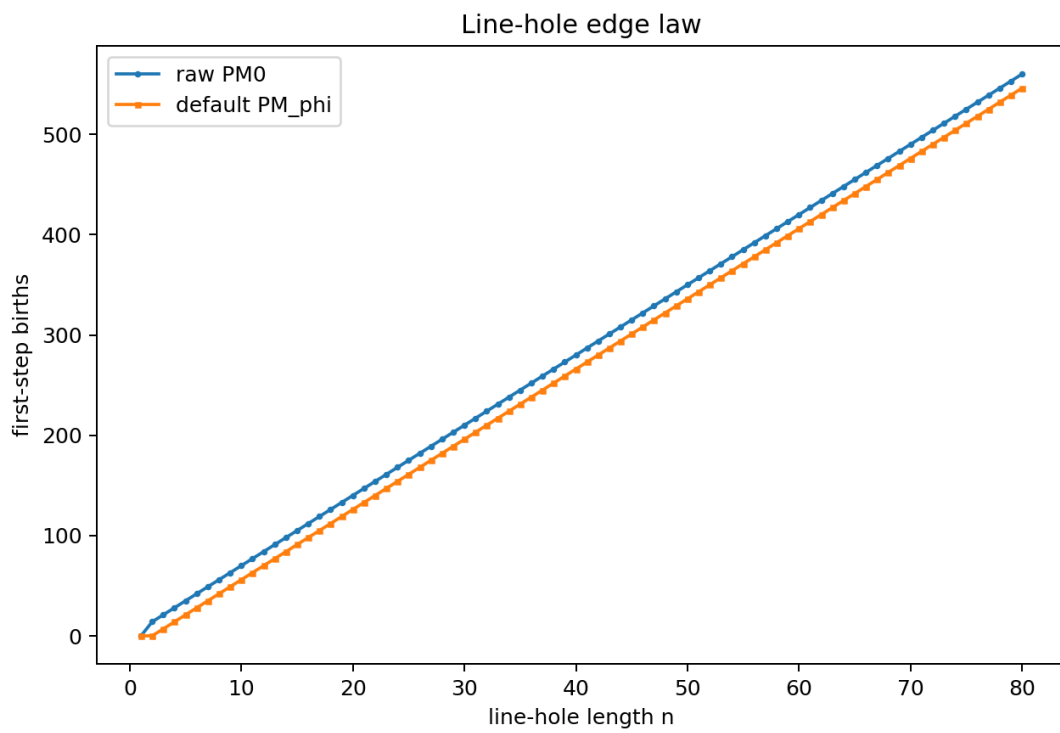


Figure 4: Regenerated raw and default prime-corrected line-hole response.

A.27 Arbitrary planar selector audit

The arbitrary planar selector audit is the most important implementation check because it tests the exact q_D/r_D selector on non-rectangular masks. Each row generates a random finite planar defect mask D , removes it from a stable sheet, runs one row update, and compares all actual first-step events against the predicted sets:

in-plane defect births: $u \in D$, $1 \leq q_D(u) \leq 3$,

survival failures/deaths: $u \notin D$, $q_D(u) \geq 4$,

adjacent normal births: $z = \pm 1$, $2 \leq r_D(u) \leq 4$.

The audit also records nonlocal first-step events. The total nonlocal event count is zero in the regenerated run.

Table 14: First ten rows from the random planar selector audit.

case	box	removed	actual E	pred E	plane B	adj B	nonlocal	ok
1	7x12	27	185	185	24	156	0	True
2	6x4	1	0	0	0	0	0	True
3	5x11	7	30	30	4	26	0	True
4	12x7	44	150	150	17	118	0	True
5	8x9	30	185	185	27	144	0	True
6	8x10	43	151	151	22	110	0	True
7	12x4	16	69	69	9	54	0	True
8	10x10	22	154	154	21	130	0	True
9	12x4	7	31	31	5	26	0	True
10	6x5	16	83	83	14	62	0	True

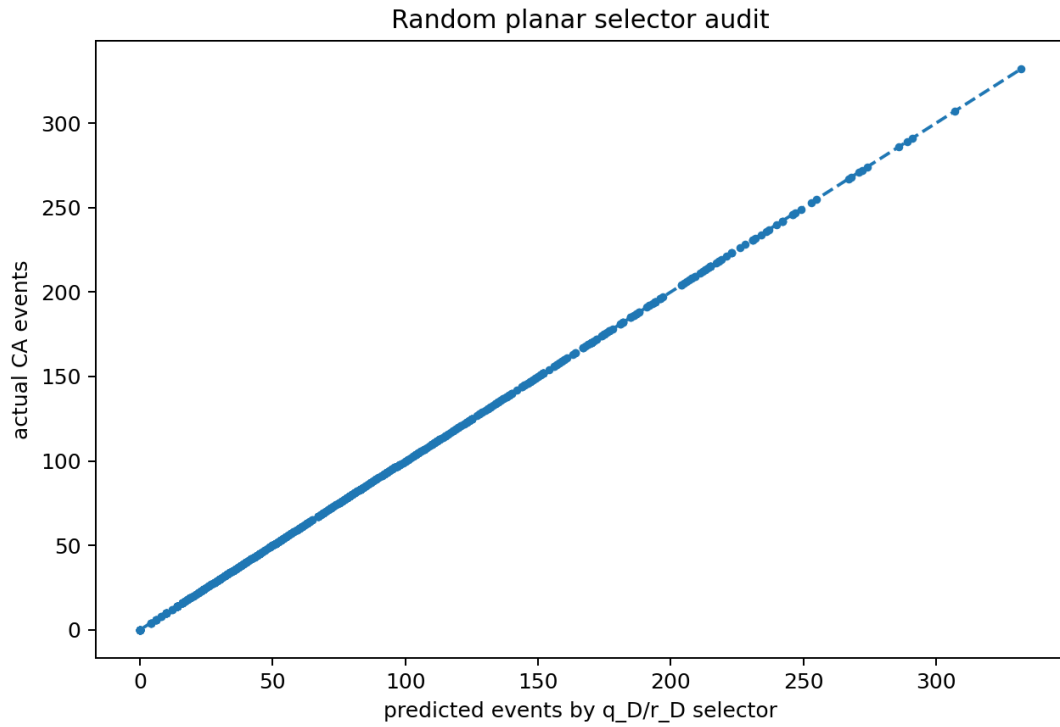


Figure 5: Actual event counts versus q_D/r_D -predicted event counts for 720 random finite planar masks.

A.28 Q/P-style rectangle reconstruction audit

This audit is included only as a boundary cross-reference to PooleQ/P-style capture. It is not the full Q/P paper. The claim tested here is the limited static rectangle reconstruction fact: area plus first-step response determines the unordered pair $\{a, b\}$ for a rectangle.

For raw PM0,

$$A = ab, \quad S = a + b = \frac{B_0 - 12}{4}.$$

For default PM_phi,

$$A = ab, \quad S = a + b = \frac{B_\phi + 4}{4}.$$

Then a and b are recovered as the integer roots of

$$t^2 - St + A = 0.$$

The regenerated audit covers $2 \leq a, b \leq 36$, giving 1225 cases.

Table 15: Representative rows from the rectangle reconstruction audit.

a	b	A	B0	Bphi	raw rec	phi rec	raw ok	phi ok
2	2	4	28	12	2x2	2x2	True	True
2	7	14	48	32	2x7	2x7	True	True
2	36	72	164	148	2x36	2x36	True	True
10	20	200	132	116	10x20	10x20	True	True
10	36	360	196	180	10x36	10x36	True	True
36	36	1296	300	284	36x36	36x36	True	True

A.29 Empirical multi-step trace table

The file `empirical_planar_relaxation_timeseries.csv` records short raw PM0 traces for selected planar rectangle holes. These rows are useful for later law discovery and for aligning the v10 atlas, but they are not theorem-level claims. They should remain in the empirical/open-problem lane until a closed multi-step law is proved.

Table 16: Representative empirical relaxation reference rows. These rows are not theorem-level evidence.

case	t	active	births	deaths	delta
2x2	0	6396	0	0	0
2x2	1	6424	28	0	28
2x2	2	6384	16	56	-12
2x2	3	6440	72	16	44
2x2	6	6608	248	240	212
2x2	12	6896	712	928	500
4x12	0	6352	0	0	0
4x12	1	6428	76	0	76
4x12	2	6348	40	120	-4
4x12	3	6512	188	24	160
4x12	6	6628	444	660	276
4x12	12	7672	1204	1436	1320
10x20	0	53624	0	0	0
10x20	1	53756	132	0	132
10x20	2	53676	96	176	52
10x20	3	54064	428	40	440
10x20	6	54372	900	1248	748
10x20	12	55296	1492	2520	1672

A.30 Evidence file schema

The full CSV and JSON artifacts are included in the companion evidence archive under **evidence/data** and **evidence/reports**. The manuscript prints only representative rows; the companion evidence archive contains the complete tables.

Evidence file schema.

`rectangle_square_channel_W_full13d_audit.csv`

rows: 841; **purpose:** Rectangles $2 \leq a, b \leq 30$; raw law, channel spectrum, prime correction, overcompensation.; **linked:** D03,D05,D05a,D07,D08,D09-D10.

`line_hole_edge_case_law_audit.csv`

rows: 80; **purpose:** Line holes $n=1..80$; raw and prime endpoint laws.; **linked:** D06.

`arbitrary_planar_selector_random_audit.csv`

rows: 720; **purpose:** Random finite planar masks; actual first-step events against qD/rD selector.; **linked:** D04,D11.

`qp_rectangle_inversion_audit.csv`

rows: 1225; **purpose:** Area plus raw/prime response recovers unordered rectangle dimensions.; **linked:** D12-D13.

`rectangle_overcompensation_criterion_audit.csv`

rows: 841; **purpose:** First-step active-count overcompensation criterion.; **linked:** D05a.

`empirical_planar_relaxation_timeseries.csv`

rows: 78; **purpose:** Multi-step traces for selected planar rectangles; empirical reference trace only.; **linked:** v10 empirical/open.

`evidence_summary.json / .csv`

rows: 16; **purpose:** Machine-readable pass/fail summary.; **linked:** release QA.

`sha256_manifest.csv`

rows: 11; **purpose:** Checksums for generated evidence files.; **linked:** reproducibility.

A.31 Evidence checksum manifest

The companion evidence archive includes a SHA-256 manifest for generated evidence artifacts. Short hashes are shown here for human inspection; the full hashes are stored in `sha256_manifest.csv`.

Short checksum list for generated evidence artifacts.

```
data/arbitrary_planar_selector_random_audit.csv
  sha256 short: 8a819cd61979f1dc; bytes: 42452.

data/empirical_planar_relaxation_timeseries.csv
  sha256 short: fe6604c3ce29df3e; bytes: 3576.

data/line_hole_edge_case_law_audit.csv
  sha256 short: 557540074695ec12; bytes: 3489.

data/qp_rectangle_inversion_audit.csv
  sha256 short: 90c7924c2dc60c1c; bytes: 46808.

data/rectangle_overcompensation_criterion_audit.csv
  sha256 short: 50ff97d6f7969302; bytes: 28370.

data/rectangle_square_channel_W_full3d_audit.csv
  sha256 short: 812208658f349ac9; bytes: 80074.

figures/line_hole_law.png
  sha256 short: df61c63dc5d169bc; bytes: 71642.

figures/planar_selector_actual_vs_predicted.png
  sha256 short: cf8616717cca5883; bytes: 58498.

figures/rectangle_birth_surface.png
  sha256 short: 4c57ca2c9e57dd1c; bytes: 46630.

figures/rectangle_channel_spectrum.png
  sha256 short: c41e9ff4adad3fb8; bytes: 56061.

figures/rectangle_overcompensation_map.png
  sha256 short: 7796427e6fc56fd6; bytes: 46414.
```

A.32 Evidence reference verifier script

The following script generated the evidence artifacts. It is included in the companion evidence archive at `evidence/scripts/poole_defect_verifier.py`. To reproduce the run from the archive root:

```
python evidence/scripts/poole_defect_verifier.py --out evidence --seed 20260620
```

Listing 1: Reference verifier.

```
#!/usr/bin/env python3
"""
PooleDefectCalculus-evidencegenerator.
License: Apache-2.0

This script regenerates finite-lattice audit artifacts for the theorem-level
first-step PooleDefectCalculus layer. It intentionally separates exact
first-step audits from empirical multi-step traces.

Model PM0:
 $x_{t+1}(i) = 1 \{ 5 \leq N_i(x_t) \leq 7 + 2x_t(i) \}$ 
where  $N_i$  is the 26-neighbor Moore count on a finite periodic cubic lattice.

Default PM_phi_evidence_layer:
For rectangle/line first-step birth channel tables, raw PM0 births with
neighbor count 7 are rejected. This is a model-tagged post-selector,
not a statement about the raw PM0 rule.
"""
from __future__ import annotations

import argparse
import csv
import hashlib
import json
import math
import os
from dataclasses import asdict, dataclass
from pathlib import Path
from typing import Dict, Iterable, List, Sequence, Set, Tuple

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

Coord2 = Tuple[int, int]
Coord3 = Tuple[int, int, int]

OFFSETS_3D: List[Coord3] = [
    (dx, dy, dz)
    for dx in (-1, 0, 1)
    for dy in (-1, 0, 1)
    for dz in (-1, 0, 1)
    if not (dx == 0 and dy == 0 and dz == 0)
]
```

```

OPEN_OFFSETS_2D: List[Coord2] = [
    (dx, dy)
    for dx in (-1, 0, 1)
    for dy in (-1, 0, 1)
    if not (dx == 0 and dy == 0)
]
CLOSED_OFFSETS_2D: List[Coord2] = [
    (dx, dy)
    for dx in (-1, 0, 1)
    for dy in (-1, 0, 1)
]

@dataclass
class RectAudit:
    a: int
    b: int
    area: int
    perimeter: int
    active0: int
    births_raw_actual: int
    births_raw_predicted: int
    deaths_actual: int
    n5_actual: int
    n6_actual: int
    n7_actual: int
    n5_predicted: int
    n6_predicted: int
    n7_predicted: int
    births_prime_actual: int
    births_prime_predicted: int
    active1_raw_actual: int
    active1_raw_predicted: int
    overcomp_actual: bool
    overcomp_predicted: bool
    raw_match: bool
    channel_match: bool
    prime_match: bool
    overcomp_match: bool

@dataclass
class LineAudit:
    n: int
    area: int
    births_raw_actual: int
    births_raw_predicted: int
    deaths_actual: int
    n5_actual: int
    n6_actual: int
    n7_actual: int
    births_prime_actual: int
    births_prime_predicted: int
    raw_match: bool

```



```

    prime_match: bool

@dataclass
class SelectorAudit:
    case_id: int
    seed: int
    box_w: int
    box_h: int
    density: float
    removed_count: int
    actual_births: int
    predicted_births: int
    actual_deaths: int
    predicted_deaths: int
    actual_plane_births: int
    predicted_plane_births: int
    actual_adjacent_births: int
    predicted_adjacent_births: int
    actual_nonlocal_events: int
    match: bool

@dataclass
class InversionAudit:
    a: int
    b: int
    area: int
    raw_births: int
    prime_births: int
    recovered_raw_a: int
    recovered_raw_b: int
    recovered_prime_a: int
    recovered_prime_b: int
    raw_match: bool
    prime_match: bool

@dataclass
class TimeseriesRow:
    family: str
    parameter: str
    t: int
    active: int
    births: int
    deaths: int
    active_delta_from_t0: int

def ensure_dir(path: Path) -> None:
    path.mkdir(parents=True, exist_ok=True)

def sha256_file(path: Path) -> str:

```

```

h = hashlib.sha256()
with path.open("rb") as f:
    for chunk in iter(lambda: f.read(1024 * 1024), b''):
        h.update(chunk)
return h.hexdigest()

def neighbor_counts(x: np.ndarray) -> np.ndarray:
    """26-neighbor_Moore_count_with_periodic_boundary_conditions."""
    counts = np.zeros_like(x, dtype=np.int16)
    for dx, dy, dz in OFFSETS_3D:
        counts += np.roll(np.roll(np.roll(x, dx, axis=0), dy, axis=1), dz, axis=2)
    return counts

def pm0_update(x: np.ndarray) -> Tuple[np.ndarray, np.ndarray]:
    """Return_(x_next, neighbor_count)."""
    n = neighbor_counts(x)
    x_next = ((n >= 5) & (n <= (7 + 2 * x))).astype(np.uint8)
    return x_next, n

def make_sheet_with_mask(mask: Set[Coord2], side: int | None = None, zsize: int = 7) ->
    Tuple[np.ndarray, int, int, int]:
    """Build_a_complete_active_sheet_with_a_finite_defect_mask_removed.

    The_returned_x_uses_axes_(x, y, z). The_sheet_is_at_z=z0.
    The_mask_is_shifted_to_the_center_of_the_sheet_to_avoid_wrap_effects.
    """
    if mask:
        max_x = max(p[0] for p in mask)
        max_y = max(p[1] for p in mask)
    else:
        max_x = max_y = 0
    if side is None:
        side = max(64, max_x + 12, max_y + 12)
    z0 = zsize // 2
    arr = np.zeros((side, side, zsize), dtype=np.uint8)
    arr[:, :, z0] = 1
    # Center local mask.
    ox = side // 2 - (max_x + 1) // 2
    oy = side // 2 - (max_y + 1) // 2
    for x, y in mask:
        arr[ox + x, oy + y, z0] = 0
    return arr, z0, ox, oy

def rectangle_mask(a: int, b: int) -> Set[Coord2]:
    return {(i, j) for i in range(a) for j in range(b)}

def first_step_events(arr: np.ndarray) -> Tuple[np.ndarray, np.ndarray, np.ndarray, np.
    ndarray]:
    nxt, counts = pm0_update(arr)

```

```

births = (nxt == 1) & (arr == 0)
deaths = (nxt == 0) & (arr == 1)
return nxt, counts, births, deaths

```

```

def audit_rectangle(a: int, b: int) -> RectAudit:
    mask = rectangle_mask(a, b)
    arr, z0, _, _ = make_sheet_with_mask(mask)
    nxt, counts, births, deaths = first_step_events(arr)
    n5 = int(np.sum(births & (counts == 5)))
    n6 = int(np.sum(births & (counts == 6)))
    n7 = int(np.sum(births & (counts == 7)))
    births_raw_actual = int(np.sum(births))
    deaths_actual = int(np.sum(deaths))
    births_raw_pred = 4 * a + 4 * b + 12
    n5_pred = 12
    n6_pred = 4 * a + 4 * b - 16
    n7_pred = 16
    births_prime_actual = n5 + n6
    births_prime_pred = 4 * a + 4 * b - 4
    side = arr.shape[0]
    active0 = side * side - a * b
    active1_actual = int(np.sum(nxt))
    active1_pred = active0 + births_raw_pred # rectangle theorem has zero deaths
    over_actual = active1_actual > side * side
    over_pred = ((a - 4) * (b - 4)) < 28
    return RectAudit(
        a=a,
        b=b,
        area=a * b,
        perimeter=2 * (a + b),
        active0=active0,
        births_raw_actual=births_raw_actual,
        births_raw_predicted=births_raw_pred,
        deaths_actual=deaths_actual,
        n5_actual=n5,
        n6_actual=n6,
        n7_actual=n7,
        n5_predicted=n5_pred,
        n6_predicted=n6_pred,
        n7_predicted=n7_pred,
        births_prime_actual=births_prime_actual,
        births_prime_predicted=births_prime_pred,
        active1_raw_actual=active1_actual,
        active1_raw_predicted=active1_pred,
        overcomp_actual=over_actual,
        overcomp_predicted=over_pred,
        raw_match=(births_raw_actual == births_raw_pred and deaths_actual == 0 and
        active1_actual == active1_pred),
        channel_match=(n5 == n5_pred and n6 == n6_pred and n7 == n7_pred),
        prime_match=(births_prime_actual == births_prime_pred),
        overcomp_match=(over_actual == over_pred),
    )

```

```

def audit_line(n: int) -> LineAudit:
    mask = rectangle_mask(1, n)
    arr, z0, _, _ = make_sheet_with_mask(mask)
    nxt, counts, births, deaths = first_step_events(arr)
    n5 = int(np.sum(births & (counts == 5)))
    n6 = int(np.sum(births & (counts == 6)))
    n7 = int(np.sum(births & (counts == 7)))
    raw_actual = int(np.sum(births))
    deaths_actual = int(np.sum(deaths))
    if n == 1:
        raw_pred = 0
        prime_pred = 0
    else:
        raw_pred = 7 * n
        prime_pred = 7 * n - 14
    prime_actual = n5 + n6
    return LineAudit(
        n=n,
        area=n,
        births_raw_actual=raw_actual,
        births_raw_predicted=raw_pred,
        deaths_actual=deaths_actual,
        n5_actual=n5,
        n6_actual=n6,
        n7_actual=n7,
        births_prime_actual=prime_actual,
        births_prime_predicted=prime_pred,
        raw_match=(raw_actual == raw_pred and deaths_actual == 0),
        prime_match=(prime_actual == prime_pred),
    )

def q_open(mask: Set[Coord2], p: Coord2) -> int:
    x, y = p
    return sum((x + dx, y + dy) in mask for dx, dy in OPEN_OFFSETS_2D)

def r_closed(mask: Set[Coord2], p: Coord2) -> int:
    x, y = p
    return sum((x + dx, y + dy) in mask for dx, dy in CLOSED_OFFSETS_2D)

def selector_prediction(mask: Set[Coord2], z0: int, ox: int, oy: int) -> Tuple[Set[Coord3], Set[Coord3], Set[Coord3], Set[Coord3]]:
    """Predict first-step events by q_D/r_D selector.

    Returns plane_births, adjacent_births, plane_deaths, all_predicted_events.
    """
    candidates: Set[Coord2] = set(mask)
    for x, y in list(mask):
        for dx, dy in CLOSED_OFFSETS_2D:
            candidates.add((x + dx, y + dy))

```

```

plane_births: Set[Coord3] = set()
plane_deaths: Set[Coord3] = set()
adjacent_births: Set[Coord3] = set()

for p in candidates:
    q = q_open(mask, p)
    r = r_closed(mask, p)
    x, y = p
    X, Y = ox + x, oy + y
    if p in mask:
        if 1 <= q <= 3:
            plane_births.add((X, Y, z0))
        else:
            if q >= 4:
                plane_deaths.add((X, Y, z0))
    if 2 <= r <= 4:
        adjacent_births.add((X, Y, z0 - 1))
        adjacent_births.add((X, Y, z0 + 1))

all_events = plane_births | adjacent_births | plane_deaths
return plane_births, adjacent_births, plane_deaths, all_events

def audit_random_selector(case_id: int, seed: int, rng: np.random.Generator) ->
SelectorAudit:
    box_w = int(rng.integers(3, 13))
    box_h = int(rng.integers(3, 13))
    density = float(rng.uniform(0.07, 0.62))
    mask: Set[Coord2] = set()
    for i in range(box_w):
        for j in range(box_h):
            if rng.random() < density:
                mask.add((i, j))
    if not mask:
        mask.add((box_w // 2, box_h // 2))
    if len(mask) == box_w * box_h:
        # Leave at least one active interior reference point so dense masks test deaths
but not full-box degeneracy only.
        mask.remove((box_w // 2, box_h // 2))

    arr, z0, ox, oy = make_sheet_with_mask(mask, side=80)
    nxt, counts, births_arr, deaths_arr = first_step_events(arr)
    pred_plane_births, pred_adj_births, pred_deaths, pred_events = selector_prediction(
    mask, z0, ox, oy)

    actual_birth_positions = set(map(tuple, np.argwhere(births_arr)))
    actual_death_positions = set(map(tuple, np.argwhere(deaths_arr)))
    actual_events = actual_birth_positions | actual_death_positions
    predicted_birth_positions = pred_plane_births | pred_adj_births
    predicted_events = predicted_birth_positions | pred_deaths

    # Event locality: the exact selector predicts all events. Anything else is nonlocal
for first-step purposes.
    nonlocal_events = len(actual_events - predicted_events)

```

```

return SelectorAudit(
    case_id=case_id,
    seed=seed,
    box_w=box_w,
    box_h=box_h,
    density=round(density, 6),
    removed_count=len(mask),
    actual_births=len(actual_birth_positions),
    predicted_births=len(predicted_birth_positions),
    actual_deaths=len(actual_death_positions),
    predicted_deaths=len(pred_deaths),
    actual_plane_births=sum(1 for p in actual_birth_positions if p[2] == z0),
    predicted_plane_births=len(pred_plane_births),
    actual_adjacent_births=sum(1 for p in actual_birth_positions if p[2] in {z0 - 1,
z0 + 1}),
    predicted_adjacent_births=len(pred_adj_births),
    actual_nonlocal_events=nonlocal_events,
    match=(actual_events == predicted_events),
)

def recover_rectangle_from_area_and_sum(area: int, s: int) -> Tuple[int, int] | None:
    disc = s * s - 4 * area
    if disc < 0:
        return None
    root = int(math.isqrt(disc))
    if root * root != disc:
        return None
    if (s - root) % 2 != 0 or (s + root) % 2 != 0:
        return None
    a = (s - root) // 2
    b = (s + root) // 2
    if a * b != area:
        return None
    return tuple(sorted((a, b))) # type: ignore[return-value]

def audit_inversion(a: int, b: int) -> InversionAudit:
    area = a * b
    raw_births = 4 * a + 4 * b + 12
    prime_births = 4 * a + 4 * b - 4
    s_raw = (raw_births - 12) // 4
    s_prime = (prime_births + 4) // 4
    rec_raw = recover_rectangle_from_area_and_sum(area, s_raw)
    rec_prime = recover_rectangle_from_area_and_sum(area, s_prime)
    target = tuple(sorted((a, b)))
    return InversionAudit(
        a=a,
        b=b,
        area=area,
        raw_births=raw_births,
        prime_births=prime_births,
        recovered_raw_a=-1 if rec_raw is None else rec_raw[0],
        recovered_raw_b=-1 if rec_raw is None else rec_raw[1],

```

```

        recovered_prime_a=-1 if rec_prime is None else rec_prime[0],
        recovered_prime_b=-1 if rec_prime is None else rec_prime[1],
        raw_match=(rec_raw == target),
        prime_match=(rec_prime == target),
    )

def run_timeseries_for_mask(mask: Set[Coord2], family: str, parameter: str, steps: int =
12) -> List[TimeseriesRow]:
    arr, z0, _, _ = make_sheet_with_mask(mask, side=max(80, len(mask) + 32), zsize=9)
    active0 = int(np.sum(arr))
    rows: List[TimeseriesRow] = []
    current = arr
    for t in range(steps + 1):
        if t == 0:
            births = 0
            deaths = 0
        else:
            # births/deaths are filled before assignment in loop below; this branch only
exists for clarity.
            births = deaths = -1
        if t == 0:
            rows.append(TimeseriesRow(family, parameter, t, int(np.sum(current)), 0, 0,
int(np.sum(current)) - active0))
        if t < steps:
            nxt, counts = pm0_update(current)
            b = int(np.sum((nxt == 1) & (current == 0)))
            d = int(np.sum((nxt == 0) & (current == 1)))
            current = nxt
            rows.append(TimeseriesRow(family, parameter, t + 1, int(np.sum(current)), b,
d, int(np.sum(current)) - active0))
        # The loop currently appends t=1..steps inside each t and also iterates those t
values, so remove duplicates.
        unique: Dict[int, TimeseriesRow] = {}
    for r in rows:
        unique[r.t] = r
    return [unique[t] for t in sorted(unique)]

def save_csv_dataclass(rows: Sequence[object], path: Path) -> None:
    if not rows:
        path.write_text("")
        return
    df = pd.DataFrame([asdict(r) for r in rows])
    df.to_csv(path, index=False)

def make_figures(out_dir: Path, rect_df: pd.DataFrame, line_df: pd.DataFrame, selector_df
: pd.DataFrame) -> None:
    ensure_dir(out_dir)

    pivot = rect_df.pivot(index="a", columns="b", values="births_raw_actual")
    plt.figure(figsize=(7.2, 5.8))
    plt.imshow(pivot.values, origin="lower", aspect="auto", extent=[pivot.columns.min(),

```

```

pivot.columns.max(), pivot.index.min(), pivot.index.max()])
plt.colorbar(label="raw_first-step_births")
plt.xlabel("b")
plt.ylabel("a")
plt.title("Rectangle_raw_first-step_birth_law")
plt.tight_layout()
plt.savefig(out_dir / "rectangle_birth_surface.png", dpi=180)
plt.close()

over = rect_df.pivot(index="a", columns="b", values="overcomp_actual")
plt.figure(figsize=(7.2, 5.8))
plt.imshow(over.values.astype(int), origin="lower", aspect="auto", extent=[over.
columns.min(), over.columns.max(), over.index.min(), over.index.max()])
plt.colorbar(label="overcompensates_intact_sheet")
plt.xlabel("b")
plt.ylabel("a")
plt.title("Rectangle_overcompensation_criterion")
plt.tight_layout()
plt.savefig(out_dir / "rectangle_overcompensation_map.png", dpi=180)
plt.close()

plt.figure(figsize=(7.2, 5.0))
plt.plot(line_df["n"], line_df["births_raw_actual"], marker="o", markersize=2, label=
"raw_PM0")
plt.plot(line_df["n"], line_df["births_prime_actual"], marker="s", markersize=2,
label="default_PM_phi")
plt.xlabel("line-hole_length_n")
plt.ylabel("first-step_births")
plt.title("Line-hole_edge_law")
plt.legend()
plt.tight_layout()
plt.savefig(out_dir / "line_hole_law.png", dpi=180)
plt.close()

plt.figure(figsize=(7.2, 5.0))
plt.plot(rect_df["perimeter"], rect_df["n5_actual"], ".", label="N5")
plt.plot(rect_df["perimeter"], rect_df["n6_actual"], ".", label="N6")
plt.plot(rect_df["perimeter"], rect_df["n7_actual"], ".", label="N7")
plt.xlabel("rectangle_perimeter")
plt.ylabel("birth-channel_count")
plt.title("Rectangle_birth-channel_spectrum")
plt.legend()
plt.tight_layout()
plt.savefig(out_dir / "rectangle_channel_spectrum.png", dpi=180)
plt.close()

plt.figure(figsize=(7.2, 5.0))
# Show predicted vs actual event counts for random planar masks.
plt.scatter(selector_df["predicted_births"] + selector_df["predicted_deaths"],
selector_df["actual_births"] + selector_df["actual_deaths"], s=8)
maxv = int(max((selector_df["predicted_births"] + selector_df["predicted_deaths"]).
max(), (selector_df["actual_births"] + selector_df["actual_deaths"]).max()))
plt.plot([0, maxv], [0, maxv], linestyle="--")
plt.xlabel("predicted_events_by_q_D/r_D_selector")

```



```

plt.ylabel("actual_CAs_events")
plt.title("Random_planar_selector_audit")
plt.tight_layout()
plt.savefig(out_dir / "planar_selector_actual_vs_predicted.png", dpi=180)
plt.close()

def write_summary(out_dir: Path, files: List[Path], summary: Dict[str, object]) -> None:
    ensure_dir(out_dir)
    (out_dir / "evidence_summary.json").write_text(json.dumps(summary, indent=2,
sort_keys=True))
    with (out_dir / "evidence_summary.csv").open("w", newline="") as f:
        writer = csv.writer(f)
        writer.writerow(["metric", "value"])
        for k, v in summary.items():
            writer.writerow([k, v])
    with (out_dir / "sha256_manifest.csv").open("w", newline="") as f:
        writer = csv.writer(f)
        writer.writerow(["relative_path", "sha256", "bytes"])
        root = out_dir.parent
        for path in sorted(files):
            if path.is_file():
                writer.writerow([str(path.relative_to(root)), sha256_file(path), path.
stat().st_size])

def main() -> None:
    parser = argparse.ArgumentParser()
    parser.add_argument("--out", type=Path, default=Path("evidence"), help="Output_
evidence_directory")
    parser.add_argument("--seed", type=int, default=20260620)
    args = parser.parse_args()

    out = args.out
    data_dir = out / "data"
    fig_dir = out / "figures"
    report_dir = out / "reports"
    ensure_dir(data_dir)
    ensure_dir(fig_dir)
    ensure_dir(report_dir)

    # Rectangle range 2..30 gives 841 cases, matching the v9 stated rectangle audit size.
    rect_rows = [audit_rectangle(a, b) for a in range(2, 31) for b in range(2, 31)]
    save_csv_dataclass(rect_rows, data_dir / "rectangle_square_channel_W_full3d_audit.csv
")
    rect_df = pd.DataFrame([asdict(r) for r in rect_rows])

    # Line audit n=1..80, matching the v9 stated line-hole audit size.
    line_rows = [audit_line(n) for n in range(1, 81)]
    save_csv_dataclass(line_rows, data_dir / "line_hole_edge_case_law_audit.csv")
    line_df = pd.DataFrame([asdict(r) for r in line_rows])

    # Random planar selector audit with 720 rows, matching the v9 stated selector audit
size.

```

```

rng = np.random.default_rng(args.seed)
selector_rows = [audit_random_selector(i + 1, args.seed, rng) for i in range(720)]
save_csv_dataclass(selector_rows, data_dir / "arbitrary_planar_selector_random_audit.
csv")
selector_df = pd.DataFrame([asdict(r) for r in selector_rows])

# Q/P-style rectangle inversion audit 2..36 gives 1225 cases, matching v9 stated
inversion size.
inversion_rows = [audit_inversion(a, b) for a in range(2, 37) for b in range(2, 37)]
save_csv_dataclass(inversion_rows, data_dir / "qp_rectangle_inversion_audit.csv")
inv_df = pd.DataFrame([asdict(r) for r in inversion_rows])

# Overcompensation criterion table is separated for easy manuscript import.
overcomp_df = rect_df[["a", "b", "area", "perimeter", "active1_raw_actual", "
overcomp_actual", "overcomp_predicted", "overcomp_match"]].copy()
overcomp_df.to_csv(data_dir / "rectangle_overcompensation_criterion_audit.csv", index
=False)

# Empirical timeseries reference traces. These are not theorem proofs.
ts_rows: List[TimeseriesRow] = []
for a, b in [(2, 2), (3, 3), (4, 4), (6, 6), (4, 12), (10, 20)]:
    ts_rows.extend(run_timeseries_for_mask(rectangle_mask(a, b), "
planar_rectangle_hole", f"{a}x{b}", steps=12))
save_csv_dataclass(ts_rows, data_dir / "empirical_planar_relaxation_timeseries.csv")

make_figures(fig_dir, rect_df, line_df, selector_df)

summary = {
    "seed": args.seed,
    "rectangle_cases": len(rect_df),
    "rectangle_raw_matches": int(rect_df["raw_match"].sum()),
    "rectangle_channel_matches": int(rect_df["channel_match"].sum()),
    "rectangle_prime_matches": int(rect_df["prime_match"].sum()),
    "rectangle_overcomp_matches": int(rect_df["overcomp_match"].sum()),
    "line_cases": len(line_df),
    "line_raw_matches": int(line_df["raw_match"].sum()),
    "line_prime_matches": int(line_df["prime_match"].sum()),
    "selector_random_cases": len(selector_df),
    "selector_random_matches": int(selector_df["match"].sum()),
    "selector_random_nonlocal_events_total": int(selector_df["actual_nonlocal_events"
].sum()),
    "qp_inversion_cases": len(inv_df),
    "qp_raw_inversion_matches": int(inv_df["raw_match"].sum()),
    "qp_prime_inversion_matches": int(inv_df["prime_match"].sum()),
    "empirical_timeseries_rows": len(ts_rows),
    "all_closed_layer_checks_passed": bool(
        rect_df["raw_match"].all()
        and rect_df["channel_match"].all()
        and rect_df["prime_match"].all()
        and rect_df["overcomp_match"].all()
        and line_df["raw_match"].all()
        and line_df["prime_match"].all()
        and selector_df["match"].all()
        and inv_df["raw_match"].all()

```

```

        and inv_df["prime_match"].all()
    ),
}

generated = list(data_dir.glob("*.csv")) + list(fig_dir.glob("*.png"))
write_summary(report_dir, generated, summary)
print(json.dumps(summary, indent=2, sort_keys=True))

if __name__ == "__main__":
    main()

```

Claim-Boundary and Language Appendices

A.33 Role of claim-boundary controls

The claim-boundary appendix does not add a new defect law. Its purpose is editorial and mathematical: every public-facing claim is assigned a class, every class is paired with safe wording, and every bridge into Q/P, continuum language, OTG language, physical interpretation, or empirical atlas language is prevented from being mistaken for a theorem.

The hardened release rule is:

A result is a theorem only when it follows from the stated local update rule, finite combinatorics, or exact algebra under its named model assumptions. A simulation is a finite audit only over the stated range. A v10 curve or 3D void trend is an empirical atlas observation unless and until an analytic law is proved. Q/P and continuum interpretations are boundaries, not hidden assumptions of the Defect Calculus.

A.34 Hardened claim-map table

The following table is the paper claim map for this version. It is intentionally redundant with earlier theorem ledgers because this table is the final editorial guardrail before v1.1-clean packaging.

ID	Tag	Result	Approved public wording	Forbidden or unsafe wording
F01	Compact hysteretic support law	EXACT	Keep as algebraic equivalence of B5-7/S5-9 and the compact hysteretic update.	Do not call the +2 term an external field.
F02	Translation and cubic symmetry	EXACT	State as discrete lattice symmetry under Moore-neighborhood-preserving transformations.	Do not imply Lorentz or continuum symmetry.
F03	Finite propagation speed	EXACT	State as Chebyshev dependency radius in the CA.	Do not equate it to physical speed of light.
F04	Eventual periodicity	EXACT	State finite-state recurrence only.	Do not imply fixed-point convergence.
F05	Non-monotonicity	EXACT COUNTEREXAMPLE	Use as a counterexample against closure-operator language.	Do not claim order-preserving dynamics.
F06	Non-reversibility	EXACT COUNTEREXAMPLE	Use as many-to-one CA behavior.	Do not claim a calibrated thermodynamic arrow.
F07	Fixed-point active-degree bounds	EXACT	Necessary fixed-point constraints only.	Do not present as full fixed-point classification.

ID	Tag	Result	Approved public wording	Forbidden or unsafe wording
F08	Sheet memory / B7 plasticity split	BOUNDARY	Use as rule-space interpretation supported by channel facts.	Do not state uniqueness/minimality.
D01	Stable active plane	EXACT	Coordinate one-cell sheet is a fixed point under stated assumptions.	Do not translate into physical spacetime proof.
D02	Surface selector	EXACT	Local dimension selector: sheet survives while point/line/bulk fail as stable flat strata.	Do not classify every possible fixed point.
D03	Square-puncture law	EXACT + AUDIT	First-step law for square holes.	Do not imply closed multi-step relaxation.
D04	One-step planar q_D/r_D defect calculus	EXACT	Mathematical core: local selector for first update of planar defects.	Do not extend to arbitrary 3D voids.
D04a	Stable pinhole gas	EXACT	Sparse pore stability corollary under stated local footprint condition.	Do not apply to dense masks/cracks.
D05	Rectangle hole law	EXACT + AUDIT	$B0(a,b)=4a+4b+12$ for $a,b \geq 2$; zero deaths first step.	Rectangle prior required.
D05a	Rectangle over-compensation criterion	EXACT	First-step active-count comparison only.	Do not state long-time growth theorem.
D06	Line-hole edge law	EXACT + AUDIT	One-cell-wide line-hole law with end-point handling.	Do not generalize to every crack.
D07	Rectangle channel spectrum	EXACT + AUDIT	Channel split $N5=12$, $N6=4a+4b-16$, $N7=16$.	Rectangle setting only.
D08	Prime-corrected rectangle law	EXACT-IN-MODEL + AUDIT	Default PM_phi removes raw $S=7$ rectangle births.	Do not present as raw PM0 law.
D09	Raw rectangle work law	EXACT-IN-DEFINITION	Discrete event/work proxy using channel weights.	Not physical energy without calibration.
D10	Prime rectangle work law	EXACT-IN-DEFINITION	Model-tagged work proxy in PM_phi .	Not raw PM0 and not calibrated energy.
D11	Model-tagged arbitrary planar selector	EXACT + AUDIT	First-step planar selector with model-specific channel indicators.	Still first-step planar only.
D12	Raw rectangle reconstruction	EXACT + AUDIT	Under rectangle prior, area plus raw response recovers side lengths up to exchange.	Births alone are ambiguous; no arbitrary-mask reconstruction.
D13	Prime rectangle reconstruction	EXACT + AUDIT	Under rectangle prior, area plus prime response recovers side lengths up to exchange.	Not dynamic Q/P capture.
E01	Raw planar cascade	EMPIRICAL	Observed in v10 atlas; law-discovery evidence.	Do not state as closed multi-step law.
E02	Prime suppression of planar cascades	EMPIRICAL	Observed comparison in v10 atlas.	Do not state as universal damping theorem.
E03	Pulse-like relaxation	EMPIRICAL	Model-selection clue from curves.	Not a derived analytic law.
E04	3D void response scaling	EMPIRICAL	Observed scaling in tested families.	Do not state as closed 3D void law.
O01	Closed multi-step planar law	OPEN	Future theorem target.	Do not promote v10 curves to theorem.
O02	Closed non-planar 3D void law	OPEN	Future theorem target.	Current status empirical only.
O03	Rule-space uniqueness	OPEN	Future theorem target.	Do not claim uniqueness.
O04	Continuum calibration	BOUNDARY / OPEN	Requires scaling limit and physical calibration.	No physical law claim in this release.
O05	Q/P dynamic-response separation	BOUNDARY	Static rectangle transducer is separate from dynamic Q/P capture.	Do not merge the full Q/P paper into this release.

A.35 Hardening by claim-status lane

Layer	Hardened status statement	Result
theorem layer	D01-D13 and F01-F07 remain exact theorem/corollary/-counterexample claims under stated CA assumptions	hardened
finite audit layer	finite sweeps remain bounded verification claims over stated domains	hardened
empirical atlas layer	v10 multi-step/3D void results remain empirical observations and candidate-law evidence only	hardened
Q/P layer	rectangle reconstruction remains under rectangle prior and is not dynamic Q/P capture	hardened
continuum/OTG layer	continuum and physics interpretations remain bridge/open-problem statements	hardened
license layer	Apache-2.0 remains the archive license	hardened

A.36 Language audit and overclaim controls

The following automated language audit is not a substitute for human peer review, but it catches the main release-risk categories. Terms such as “gravity”, “continuum”, “quantum”, or “energy” are not banned; they must remain inside boundary sections unless a theorem, finite audit, or calibrated derivation is supplied.

Risk category	Count	Control action
physical law overclaim	1	Quarantine in boundary/open/interpretation lane if present; do not use as theorem language.
spacetime/gravity bridge terms	8	Quarantine in boundary/open/interpretation lane if present; do not use as theorem language.
quantum/QP bridge terms	29	Quarantine in boundary/open/interpretation lane if present; do not use as theorem language.
thermodynamic/energy calibration terms	49	Quarantine in boundary/open/interpretation lane if present; do not use as theorem language.
continuum bridge terms	28	Quarantine in boundary/open/interpretation lane if present; do not use as theorem language.
uniqueness terms	6	Quarantine in boundary/open/interpretation lane if present; do not use as theorem language.
absolute proof phrasing	0	Quarantine in boundary/open/interpretation lane if present; do not use as theorem language.

A.37 Theorem-level statements that are safe to foreground

The theorem-level layer consists of the following foreground results, provided the finite-lattice Poole-rule assumptions are stated:

1. the compact hysteretic support law for $B5-7/S5-9$;
2. stable coordinate sheets and surface selection by local Moore-neighbor counts;
3. first-step planar defect calculus using q_D and r_D ;
4. rectangle, square, and line-hole first-step laws in their stated domains;
5. rectangle channel spectra and prime-corrected laws inside the named PM_ϕ model;
6. discrete irreversible-work proxies as definitions over channel weights;
7. rectangle reconstruction from area plus response under a rectangle prior.

A.38 Statements that must stay bounded

The following statements remain bounded and are not theorem-level claims:

1. **v10 atlas:** may be described as reproducible empirical evidence for candidate multi-step and void-response laws; it may not be called a closed analytic theorem.
2. **Q/P capture:** may be referenced as an application boundary and as static rectangle-transducer evidence; it may not be presented as arbitrary-mask or dynamic Q/P reconstruction.
3. **Continuum and OTG:** may be listed as future bridge work; this document does not prove spacetime, gravity, dark energy, or a continuum metric theory.
4. **Rule-space uniqueness:** may be listed as an open problem; this document does not prove that the Poole rule is unique or minimal among all possible rules.
5. **Thermodynamic language:** the work laws are discrete event-weighting laws unless an external calibration to physical energy is supplied.